
HPE Career Preview Program (CPP3)

JECRC University, Jaipur

February 2026 – June 2026

SOFTWARE DESIGN DOCUMENT

Agentic Bug Triage & Routing System

A multi-agent pipeline for real-time cross-system bug aggregation, correlation, and unified triage

Document Type	Software Design Document
Version	1.0 - Final
Project Period	February 2026 – June 2026

PROJECT TEAM

Disha Jain · Anuj Modani · Shivansh Gaur · Om Jain · Pulkit Jain

HPE INDUSTRY MENTORS

Mr. Madhukar Rupakumar · Mr. Bharath Narayan · Mr. Rajesh P E · Mr. Divakar Padiyar

College Mentor

Mr. Aneesh Mishra

Assistant Professor · Department of Computer Science · JECRC University, Jaipur

Abstract

Large software organisations like HPE run multiple bug tracking systems simultaneously. A single customer complaint might generate tickets in JIRA, Bugzilla, and GitHub Issues at the same time, filed by different teams, using different terminology, with no cross-references between them. Finding these connections manually takes engineers anywhere from a few minutes to hours per incident.

This document describes the Agentic Bug Triage and Routing System. The system connects to bug tracking sources across multiple tracker types, shows all bugs in a single live dashboard, and when an engineer triggers a triage, runs a four-agent AI pipeline that fetches the full bug context, searches every other connected system for related issues, retrieves relevant documentation from Confluence, and produces a structured AI analysis - all within a few seconds.

Three hard constraints were agreed with HPE mentors from the start. The platform is strictly read-only and never writes to any external system. Bug content from external systems is never stored permanently - only the AI's own analysis output is retained. The AI's severity classification must be deterministic so the same bug always produces the same result.

Table of Contents

1. Introduction	6
1.1 The Problem Being Solved	6
1.2 What This System Does	6
1.3 Project Objectives	7
1.4 Scope and Boundaries	7
1.5 Key Terms and Definitions	8
2. Feasibility Study	9
2.1 Technical Feasibility	9
2.2 Operational Feasibility	9
2.3 Time Feasibility	9
2.4 Risk Assessment	9
3. Requirements Specification	11
3.1 Functional Requirements	11
3.2 Non-Functional Requirements	12
3.3 Use Cases	12
4. System Architecture	13
4.1 Overview	13
4.2 How the Layers Interact	15
4.3 Design Decisions	16
4.4 Technology Stack	16
4.5 Internal Module Structure	17
4.6 Communication Protocols	22
4.7 Infrastructure	23
5. Database Design	25
5.1 Entity Relationship Diagram	25
5.2 Table Descriptions	25
5.3 What Is and Is Not Stored	26
6. Connector Architecture	28
6.1 The Adapter Pattern	28
6.2 BaseConnector Interface	29
6.3 TicketData - Universal Bug Representation	30
6.4 All Connector Implementations	30
6.5 ConnectorRegistry	31
6.6 Severity Normalisation	31
6.7 Dynamic Configuration via Token Settings	32
7. Sequence Diagrams	33
SD 1 Live Bug List Overview	33
SD 1.1 Initial Page Load - Cold Cache	34
SD 1.2 Warm Cache Retrieval	35
SD 1.3 Search Using Warm Cache	36
SD 1.4 Severity Filter - Partial Cache Miss	37
SD 2 Complete Triage Pipeline	38

SD 2.1 System Architecture Overview	39
SD 2.2 Request Entry & Event Trigger	39
SD 2.3 Context Fetch Pipeline	40
SD 2.4 Parallel Intelligence Layer	41
- CrossSystemFetchAgent(Phase 2a)	42
- EnrichmentAgent (Phase 2b)	42
SD 2.5 AI Synthesis & Structured Output	43
SD 2.6 Cleanup & Reliability	44
SD 3 Connector Configuration (Token Settings)	45
SD 4 Incremental Change Detection - Hybrid Approach	46
SD 5 Crash Recovery & Fault Tolerance	47
- Orchestrator -A Phase(Normal Execution Until Crash)	48
- Orchestrator-B Phase(Recovery and Resumption)	48
8. Agent Pipeline Design	49
8.1 Pipeline Structure and Phase Ordering	49
8.2 Phase 0 - Triggering the Pipeline	50
8.3 Phase 1 - ContextFetchAgent	50
8.4 Phase 2a - CrossSystemFetchAgent	51
8.5 Phase 2b - EnrichmentAgent	51
8.6 Phase 3 - AISynthesisAgent	52
8.7 Cleanup and Crash Recovery	53
9. Implementation	54
9.1 Backend Implementation	54
9.2 Kafka Event Processing	54
9.3 Redis Caching Layer	55
9.4 AI Agent Implementation Pattern	55
9.5 Reliability and Graceful Degradation	56
10. API Design	57
10.1 Authentication Endpoints	57
10.2 Bug Discovery and Triage Endpoints	57
10.3 History, Metrics, and Settings Endpoints	57
10.4 Cases Endpoints	58
11. Frontend Design	58
11.1 Technology Overview	58
11.2 Pages	59
11.3 WebSocket Connection Management	59
12. Security Design	63
12.1 JWT Authentication	63
12.2 Role-Based Access Control	63
12.3 OpenBAO Token Vault	63
12.4 Data Privacy by Architecture	63
13. Proof of Concept Demonstration Plan	64
13.1 Demonstration Scenarios	64
14. Data Sources	65

15. Conclusion and Future Work	67
15.1 What This System Delivers	67
15.2 Current Limitations	67
References	68
Appendix A - Glossary	70
Appendix B - Full API Reference	71
Appendix C - Environment Variables	72
Appendix D - Database Schemas	73

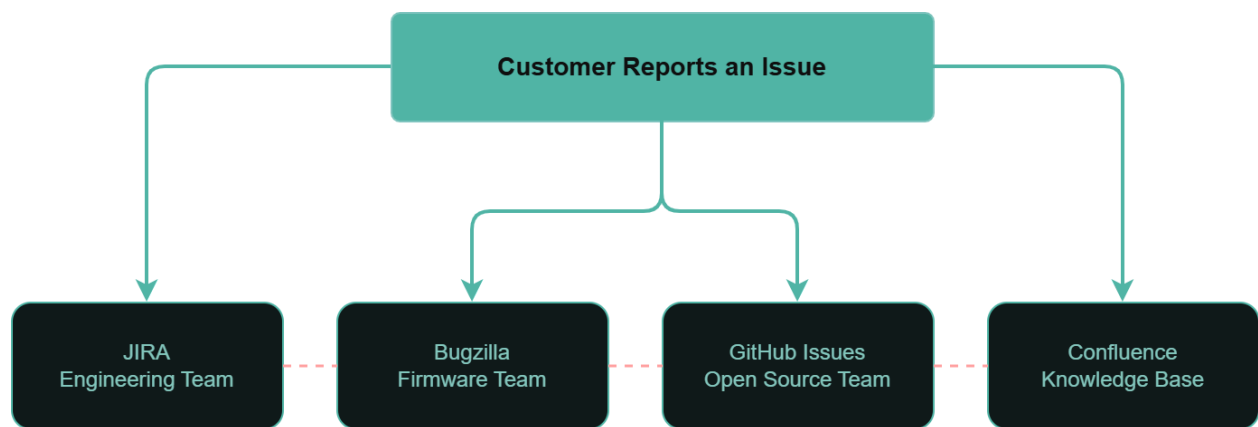
1. Introduction

1.1 The Problem Being Solved

When a software product has a bug, multiple things happen at once. Customer support opens a case in their portal. Engineering creates a ticket in JIRA or Bugzilla. If the product has an open-source component, someone might file an issue on GitHub. A knowledge base article from a previous similar incident might already exist in Confluence. All of these artefacts describe the same problem but live in completely separate systems, written by different people, using different vocabulary, with no links between them.

The engineer responsible for fixing the bug has to check all of these systems manually, one by one, tab by tab. They have to know which systems to check, know how to search each one, and mentally connect the dots between unrelated-looking tickets. This process is slow, inconsistent, and entirely dependent on each individual engineer's knowledge of the organisation's tooling. When that engineer leaves, the institutional knowledge of those connections goes with them.

At HPE this problem is compounded by the scale of the product portfolio. Different product lines use different JIRA instances. Firmware teams use Bugzilla while software teams use JIRA. Some components are open source and tracked on GitHub. The same infrastructure bug might be filed as STOR-441 in one system, BZ#1899234 in another, and issue #22471 in a third, with no automatic connection drawn between them.



No cross-references between systems. Engineer must search each one manually.

[Figure 1 - One customer incident generating fragmented tickets across four disconnected systems]

1.2 What This System Does

The Agentic Bug Triage and Routing System is a web application that sits on top of all these bug tracking systems as a unified intelligence layer. It connects to all configured bug tracking sources simultaneously, pulls their open bugs into a single live list, and when an engineer wants to investigate a specific bug, runs a four-stage AI pipeline to do the investigative work automatically.

The pipeline fetches everything known about the bug from its source system, searches every other

connected system for related bugs using LLM-generated semantic search queries, searches the Confluence knowledge base for relevant documentation using an iterative reasoning strategy, and produces a structured triage output containing severity, root cause, recommended actions, and both an engineer-facing. All of this appears in the browser panel by panel, progressively, as each stage finishes. The total time from clicking Triage to seeing all four panels is in seconds.

1.3 Project Objectives

1. Build a single unified bug list from all connected systems, auto-discovered on page load without manual ID entry.
2. Implement a four-phase AI pipeline completing in just seconds.
3. Use LLM-generated semantic queries to find related bugs across systems - understanding the underlying technical concept, not just matching keywords.
4. Connect to real Atlassian Confluence for knowledge base enrichment using a three-round iterative search.
5. Enforce data sovereignty: bug content from external systems is never stored permanently. Only AI-generated outputs and internal bug ids are retained.
6. Support crash recovery - a mid-pipeline crash must resume from the last completed step on restart.
7. Allow new bug tracking systems to be connected via a UI form with no code changes required.
8. Implement JWT authentication so only authenticated engineers can access triage results and system configuration.

1.4 Scope and Boundaries

In scope:

- Live bug list from all configured public open-source trackers
- Four-phase AI triage pipeline with WebSocket streaming and crash recovery
- Real Atlassian Confluence Cloud integration for knowledge base enrichment
- Customer Portal connector showing linked customer cases in Panel 1
- JWT authentication for engineer access
- Dynamic connector configuration via Token Settings UI with no code changes
- Incremental change detection showing what changed since last triage

Out of scope:

- Write operations to any external system - strictly read-only
- Connection to HPE internal systems - this proof of concept uses public data sources

1.5 Key Terms and Definitions

Term	Meaning
Bug / Ticket	A written problem report submitted to a bug tracking system. Has a unique ID, title, description, severity, and status.
Source	One external bug tracking system connected to the platform. Stored as a row in

	source_registry. Eg. Github, Jira, Bugzilla
Connector	A Python class that translates one external system's API responses into the common TicketData format.
ConnectorRegistry	Runtime module that loads all enabled connectors from the database and provides them to agents on request.
TicketData	The common Python dataclass that all connectors normalise their API responses into. Same schema regardless of source.
source_registry	PostgreSQL table with one row per connected system. The single source of truth for what is connected.
pipeline_context	PostgreSQL row that stores working state between agent phases. Acts as a crash-safe checkpoint. Deleted on completion.
audit_log	PostgreSQL table recording the AI's own outputs after each triage. Never contains bug content from external systems.
SynthesisOutput	The Pydantic-validated output schema of AISynthesisAgent: severity, root cause, recommended actions, affected components, engineer summary, confidence.
Case ID	UUID generated per triage request. Ties together the Kafka event, pipeline_context row, WebSocket channel, and Redis result cache.
ReAct	Reason and Act. A technique where an LLM decides what tool to call, calls it, reads the result, and decides whether to continue.
temperature=0	LLM setting making output fully deterministic. The same input always produces the same output. Required for AISynthesisAgent.
Data Sovereignty	The requirement that bug content from external systems is never stored permanently in this system's own database.
Soft Delete	Setting enabled=false on a row rather than removing it, preserving the row for audit purposes.
ADF	Atlassian Document Format. The nested JSON structure Atlassian Cloud JIRA uses for description fields. Requires recursive text extraction.

2. Feasibility Study

2.1 Technical Feasibility

Every technology in the stack is mature, well-documented, and proven in production systems. Python's asyncio model is the right fit for a system that spends most of its time waiting for external HTTP responses. FastAPI's native async support and WebSocket handling make the streaming panel delivery pattern straightforward. Redis pub/sub is a standard pattern for bridging background workers with WebSocket connections. Kafka's uncommitted offset mechanism is exactly what crash recovery requires.

The Groq API provides fast LLM inference - the Llama 3.3 70B model responds quickly for the short prompts used by ContextFetchAgent and CrossSystemFetchAgent. JIRA, Bugzilla, and the GitHub repositories are all publicly accessible without credentials, meaning the system can be demonstrated without any special access arrangements.

2.2 Operational Feasibility

The system requires no changes to any existing workflow. Engineers keep filing bugs in JIRA, GitHub, or Bugzilla exactly as they do today. The platform only reads from those systems and never writes. If the platform is shut down, nothing changes in any source system. Adoption by one engineer has no effect on colleagues who do not use the platform.

2.3 Time Feasibility

The project runs from 19 February 2026 to the end of June 2026. A phased approach ensures demonstrable output at each phase boundary: infrastructure and connectors first, then the agent pipeline, then the frontend and integration, then testing and documentation. Mentor reviews happen at phase boundaries rather than only at submission.

2.4 Risk Assessment

Risk	Likelihood	Impact	Mitigation
Groq API rate limit hit during live demo	Medium	High	Tenacity library applies exponential backoff. Identical prompts are cached in Redis so repeat triages of the same bug do not consume API calls.
External bug tracker changes its API schema	Low	Medium	The connector normalisation layer absorbs schema changes. Only the affected connector class needs updating.
Kafka unavailable in development environment	Low	Low	ENABLE_LOCAL_PIPELINE_FALLBACK env var runs the orchestrator as an asyncio task directly inside FastAPI. No Kafka dependency for development.
LLM returns malformed JSON for synthesis	Medium	Medium	Two-attempt retry: the second attempt adds an explicit JSON schema block to the prompt. If both fail, a deterministic keyword classifier provides the final fallback.
WebSocket connects after panels start publishing	Medium	High	Each panel is stored as a persistent Redis key in addition to pub/sub. WebSockets replay stored panels on connect

			before listening for new ones.
Atlassian Confluence API rate limit	Low	Low	EnrichmentAgent makes at most three API calls per triage. Free Atlassian tier allows 10,000 calls per month.

3. Requirements Specification

3.1 Functional Requirements

ID	Requirement	Priority
FR-01	Display a live unified bug list from all connected sources without requiring manual bug ID entry.	Must Have
FR-02	Support filtering by severity, source system, and status, and searching by bug ID or keyword.	Must Have
FR-03	Execute a four-phase pipeline: ContextFetch alone in Phase 1, CrossSystemFetch and Enrichment in parallel in Phase 2, AISynthesis in Phase 3.	Must Have
FR-04	Stream results via WebSocket - each panel appears as its agent finishes, not all at once at the end.	Must Have
FR-05	CrossSystemFetchAgent shall use the full primary ticket to generate a search query via LLM, then fire it against all other connected systems simultaneously.	Must Have
FR-06	CrossSystemFetchAgent shall score every candidate ticket against the primary using all available fields.	Must Have
FR-07	EnrichmentAgent shall search real Confluence using a multi-round iterative strategy informed by previous round results.	Must Have
FR-08	AISynthesisAgent shall produce validated structured output: severity (P0–P3), root cause, recommended actions, affected components, confidence, engineer summary.	Must Have
FR-09	AISynthesisAgent shall use temperature=0 so the same bug always receives the same classification.	Must Have
FR-10	Detect whether a previously triaged bug has changed since the last triage and show specifically what changed.	Must Have
FR-11	New source systems shall be connectable via a UI form without code changes, live immediately after saving.	Must Have
FR-12	Authenticated engineers see full technical triage results including similarity scores, LLM reasoning, and source ticket links.	Must Have
FR-13	A mid-pipeline crash must resume from the last completed step on restart without duplicate API calls.	Must Have
FR-14	Panel 1 shall display linked customer cases retrieved from the Customer Portal connector.	Should Have
FR-15	A History page shall show all past triages from the audit log with the ability to view previous results.	Should Have

3.2 Non-Functional Requirements

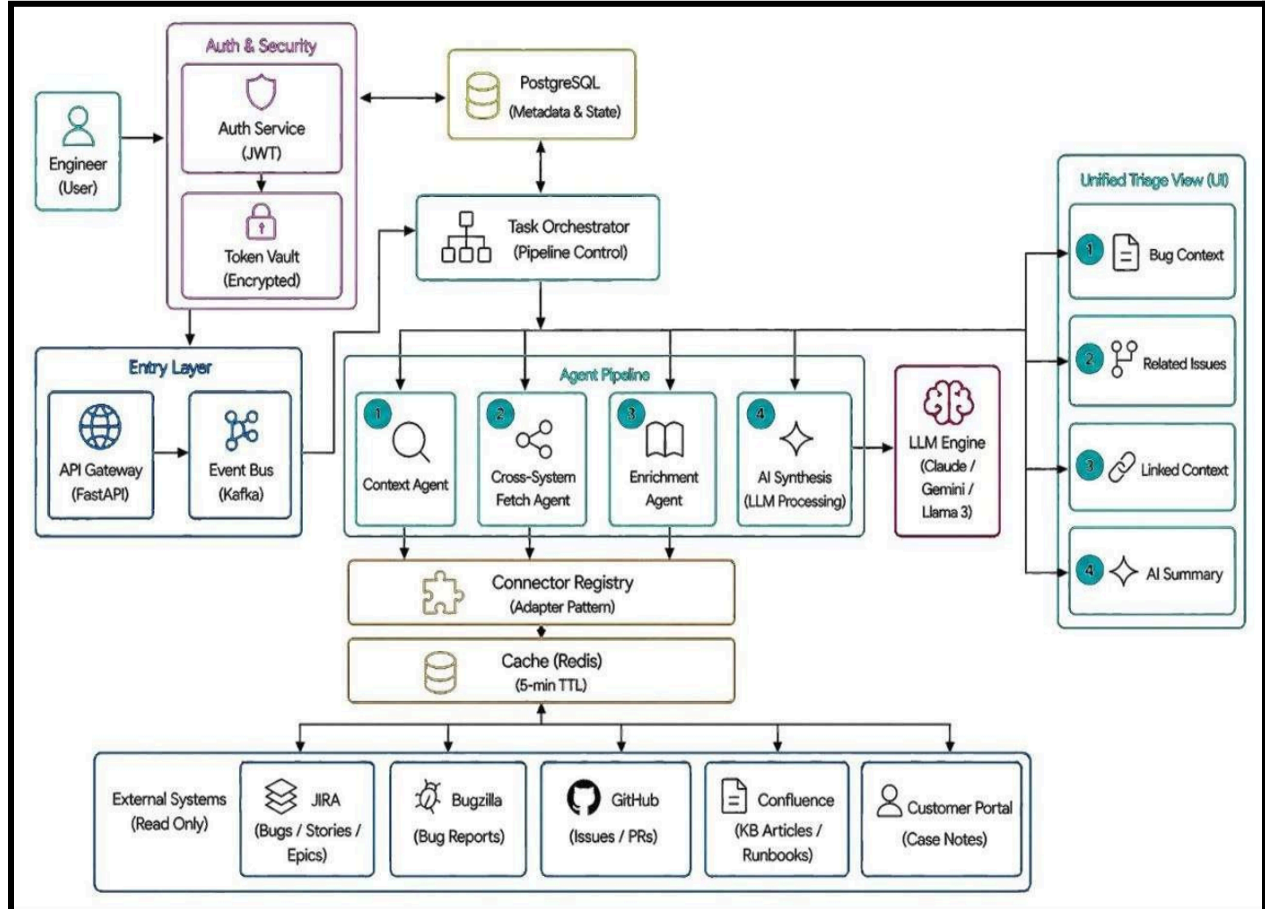
ID	Requirement	Target
NFR-01	End-to-end triage completion	P50 under 25 seconds, P95 under 30 seconds
NFR-02	Bug list - warm cache response	Under 200 milliseconds
NFR-03	Bug list - cold cache, all connectors	Under 15 seconds
NFR-04	Data sovereignty	Zero bug content in any permanent storage table
NFR-05	AI determinism	100% - enforced by temperature=0
NFR-06	Connector failure isolation	A timeout in one connector must not affect results from others
NFR-07	Crash recovery	Resume from last completed step, not pipeline start
NFR-08	WebSocket delivery under race conditions	Panels delivered even if WebSocket opens after they start publishing

3.3 Use Cases

Use Case	Actor	What Happens
UC1 - Login	Engineer	Engineer submits email and password. The system returns a JWT. Every subsequent request carries this token. Expired tokens return 401.
UC2 - Connect a New System	Engineer	Engineer fills the Add Connection form. The system makes a live test API call. If it passes, the connection is saved and is live. If it fails, nothing is saved.
UC3 - View Bug List	Engineer	Page loads, all configured sources queried in parallel with Redis cache checked first, results merged and sorted by severity, paginated at 10 per page.
UC4 - Triage a Bug	Engineer	Click Triage. System returns 202 immediately. WebSocket streams four panels progressively as agents finish.
UC5 - Check for Changes	Engineer	Expand a previously triaged bug row. System checks audit_log and makes one lightweight connector call, then shows either no change or a specific change delta.

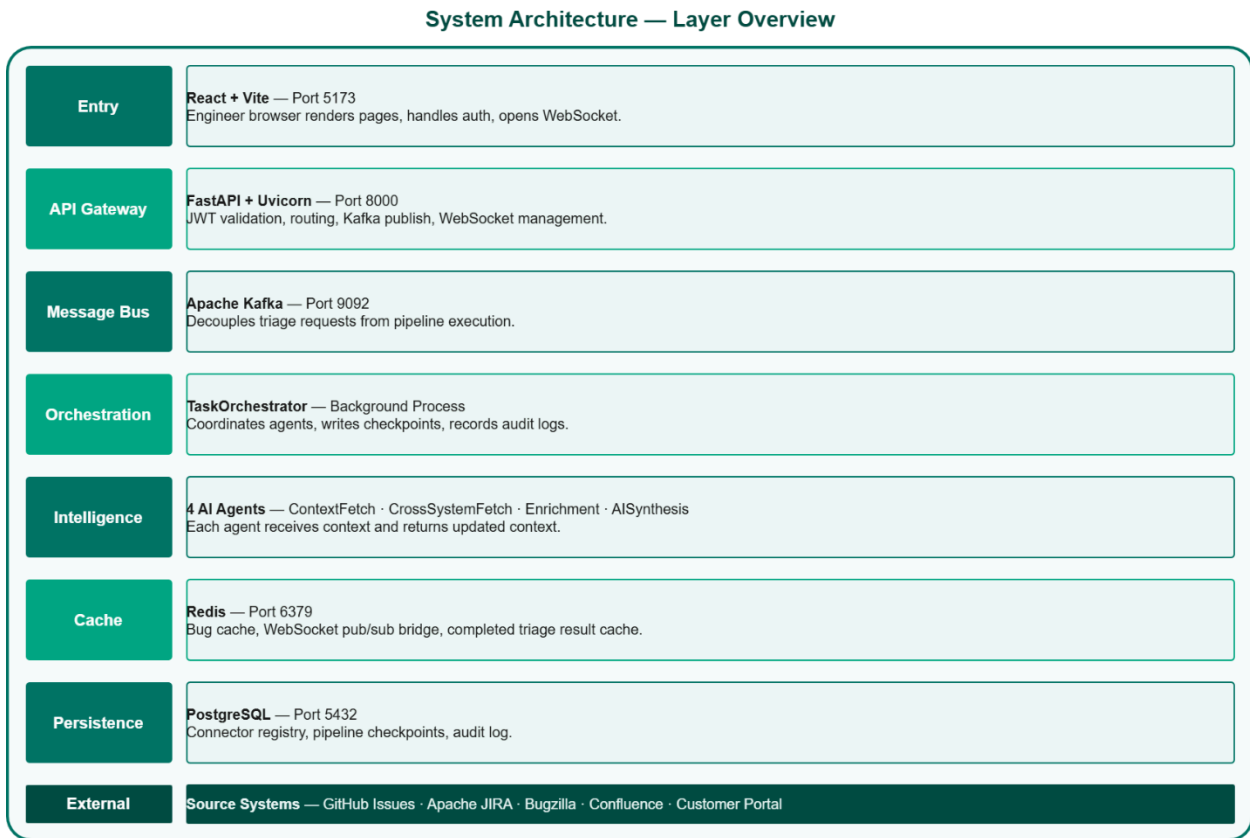
4. System Architecture

4.1 Overview



[Figure 1 - High level Design]

The system is built in six layers between the engineer's browser and the external bug tracking systems. The browser talks only to the API gateway. The gateway publishes to Kafka and reads/writes Redis but never touches the orchestrator or external systems directly. The orchestrator runs agents and manages PostgreSQL state. Agents call external systems through connectors. This separation means each layer can be tested, replaced, and scaled independently.

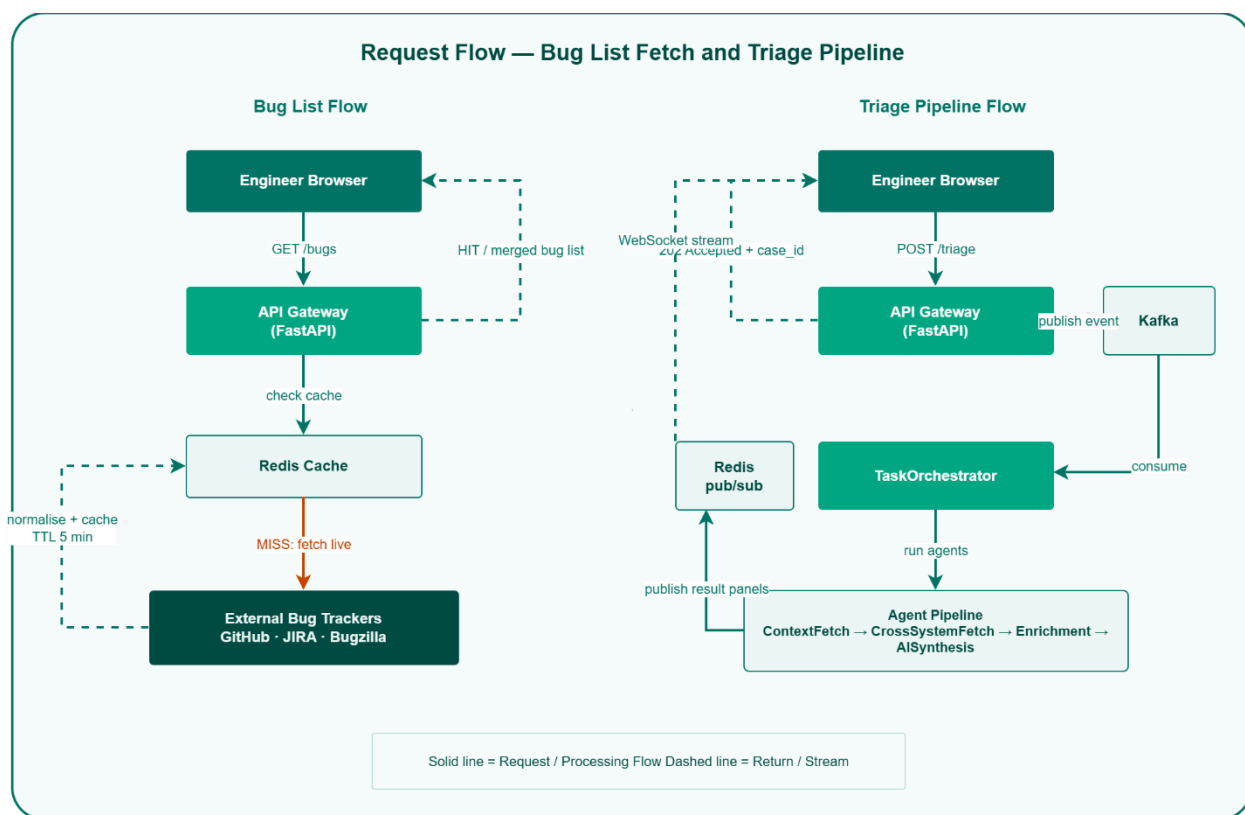


[Figure 2 - Six-layer architecture from browser to external sources]

Layer	Component	What It Does
Entry	React + Vite, port 5173	Renders all pages. Open WebSocket connections. Sends API requests with JWT headers.
Gateway	FastAPI + Uvicorn, port 8000	Validates JWT on every request. Routes HTTP requests. Publishes triage events to Kafka. Manages WebSocket subscriptions via Redis.
Message Bus	Apache Kafka, port 9092	Buffers triage requests between gateway and orchestrator. Uncommitted messages are redelivered on restart.
Orchestration	TaskOrchestrator, background	Consumes Kafka events. Runs agents in phase order. Writes pipeline checkpoints. Writes AI observations to audit_log.
Intelligence	4 Agent Classes	ContextFetchAgent, CrossSystemFetchAgent, EnrichmentAgent, AISynthesisAgent. Each receives a context

		dict and returns an updated one.
Cache	Redis, port 6379	Bug list cache (5-minute TTL). Panel pub/sub and persistent keys. Completed triage results (1-hour TTL).
Persistence	PostgreSQL, port 5432	Connector config, pipeline checkpoints (transient), AI observation log, knowledge base articles.
External	11 Source Systems	GitHub Issues (5 repos), Apache JIRA (5 projects), Mozilla Bugzilla, HPE Confluence, Customer Portal. Read-only. Never modified.

4.2 How the Layers Interact



[Figure 3 - Request flow for bug list fetch and for triage pipeline]

The bug list flow is a cache-aside pattern. The gateway creates one task per connector and runs all configured connectors concurrently. Each task checks Redis first. On a miss, it calls the external API, normalises responses to TicketData, and writes to Redis with a 2-minute TTL. A background task continues fetching additional pages after the HTTP response returns, so the cache gets richer without blocking the engineer.

The triage flow is event-driven. The gateway publishes to Kafka and returns 202 immediately. The browser opens a WebSocket and subscribes to a Redis pub/sub channel keyed by the case ID. The orchestrator picks up the Kafka event and starts running agents. As each agent finishes, its panel output is stored as a persistent Redis key and published to the channel. If the WebSocket connects after some panels have already been published, it replays the stored keys before listening for new messages. This

replay mechanism is the solution to the race condition between a fast orchestrator and a slow browser connection.

4.3 Design Decisions

Pattern	Where Used	Reason
Adapter Pattern	All connectors	GitHub, JIRA, and Bugzilla have completely different APIs. The pattern means the pipeline only ever sees TicketData. Adding a new tracker means adding one class with no other changes.
Cache-Aside	GET /bugs endpoint	External APIs are slow and have rate limits. Caching means concurrent users all share the same warmed cache, and the system behaves well under load.
Event-Driven via Kafka	POST /triage	The pipeline takes a few seconds. A synchronous request would hold the connection open. Kafka also provides crash recovery via uncommitted offsets for free.
ReAct (Reason + Act)	EnrichmentAgent	A single-round Confluence search often misses relevant articles because the first query is not specific enough. Iterative rounds produce consistently better results.
Checkpoint-Resume	pipeline_context table	Without checkpointing, a crash at eight seconds wastes all the work already done. With checkpointing, the pipeline continues from where it left off.
Pub/Sub with Persistent Replay	Redis + WebSocket	Pub/sub has no history - a message published before any subscriber is listening is permanently lost. Persistent Redis keys fix this.

4.4 Technology Stack

Technology	Version	Role	Why Chosen
Python	3.11+	Primary language	Native async/await, strong AI ecosystem, Pydantic integration.
FastAPI	0.111	API gateway	Async-native, automatic OpenAPI generation, built-in WebSocket.
Pydantic v2	2.7	Output validation	Validates the LLM's JSON against SynthesisOutput schema. Validation failures trigger the retry chain.
SQLAlchemy 2	2.0	ORM	Async session support with connection pooling for all PostgreSQL operations.
aiokafka	0.10	Kafka client	Async producer (gateway) and consumer (orchestrator) in one library.
redis-py async	5.0	Redis client	Async cache reads/writes and pub/sub in the same library.

Groq SDK	0.8	LLM inference	Llama 3.3 70B model with sub-500ms response times for short prompts.
httpx	0.27	External HTTP	Async client for all connector calls. Per-request timeout configuration.
python-jose	3.3	JWT	Generates and validates HS256 tokens signed with an environment-variable secret.
tenacity	8.3	Retry logic	Exponential backoff on LLM and connector calls.
React 18	18+	Frontend	Component model, hooks for state, React Router for navigation.
Vite 5	5+	Build tool	Hot module replacement for fast development. Clean production builds.
PostgreSQL 15	15 Alpine	Database	ACID, JSONB for flexible structured storage in pipeline_context and audit_log.
Redis 7	7 Alpine	Cache and pub/sub	In-memory speed for cache reads. Pub/sub for WebSocket panel bridging.
Apache Kafka	Confluent 7.5	Event bus	At-least-once delivery. Consumer group offset management for crash recovery.
Docker Compose	v2	Local infrastructure	All five services start with one command.

4.5 Internal Module Structure

The backend is organised into clearly separated modules. Each module has one responsibility and one location. This structure means a developer can locate any component in the codebase without searching.

Group 1 - API Gateway Core · api_gateway/		
Module	File Location	Responsibility
<code>config</code>	api_gateway/config.py	Loads all env vars at import time: JWT settings, Postgres URL, Redis URL + TTLs, Kafka bootstrap/topic/consumer-group, local pipeline fallback flag, Groq API key and model name. Single source of truth for runtime configuration.
<code>auth</code>	api_gateway/auth.py	User dataclass; verify_password + authenticate_user (pbkdf2_sha256 check against user_roles table); create_access_token (HS256 JWT); FastAPI dependencies get_current_user (OAuth2 bearer) and require_role (role-check factory). Runs before every protected handler.
<code>kafka_client</code>	api_gateway/kafka_client.py	kafka_lifespan async context manager: creates AIOKafkaProducer on startup, stops it on shutdown, stores it on app.state.kafka_producer. publish_triage_request serialises payload to JSON and publishes to the triage.requests topic.
<code>websocket_manager</code>	api_gateway/websocket_manager.py	ConnectionManager maps case_id to WebSocket. subscribe_and_forward subscribes to Redis pubsub channel

		ws:{case_id}, replays all stored panels for late-joiners, then forwards live messages. Sends a heartbeat every 15s.
main	api_gateway/main.py	FastAPI app entry point. Lifespan context manager starts Kafka producer, calls ensure_runtime_schema, starts Kafka consumer task, and a 60-second background bug-list ingestion loop (Redis lock:ingest:all). Registers CORS, HTTP-logging middleware, mock stubs, and all routers.
__init__	api_gateway/__init__.py	Package initializer.

Group 2 - Routes · api_gateway/routes/

Module	File Location	Responsibility
__init__	api_gateway/routes/__init__.py	Re-exports auth_router, cases_router, triage_router, settings_router for registration in main.py.
auth_routes	api_gateway/routes/auth_routes.py	POST /auth/login, GET /auth/users, POST /auth/users, DELETE /auth/users/{email} - all delegate to auth_service / user_service. GET /auth/me remains inline (single serialisation, no business logic).
cases_routes	api_gateway/routes/cases_routes.py	Thin handlers for GET /bugs, GET /bugs/{bug_id}/status, GET /metrics, GET /cases/{case_id}, GET /history/triage. Each delegates entirely to bug_service, metrics_service, case_service, or triage_service.
triage_routes	api_gateway/routes/triage_routes.py	POST /triage extracts kafka_producer from app.state and delegates to triage_service.start_triage. GET /triage/{case_id}/result delegates to triage_service.get_triage_result. WS /triage/{case_id}/stream manages WebSocket lifecycle and JWT decode directly.
settings_routes	api_gateway/routes/settings_routes.py	GET/POST /settings/connections and POST /settings/connections/{source_id}/test delegate to connection_service. PUT and DELETE remain partially inline. Legacy PUT /connections/{source_id} uses raw sql_update(SourceRegistry) - not yet refactored.

Group 3 - Service Layer · api_gateway/services/

Module	File Location	Responsibility
auth_service	api_gateway/services/auth_service.py	login(email, password) - delegates to authenticate_user, calls create_access_token, returns a plain dict serialised by the route's response_model=LoginResponse.
user_service	api_gateway/services/user_service.py	list_users(), create_user(email, password, role, display_name) (uniqueness check + pbkdf2_sha256 via create_user_repo), delete_user(email). All open AsyncSessionLocal directly and commit.
bug_service	api_gateway/services/bug_service.py	get_bugs(...) - paginated, filtered, sorted bug list with Redis-first cache, connector fallback, and 5-step batch DB grouping via assemble_grouped_bug_list. get_bug_status(bug_id) - DB lookup → two-strategy connector resolution → lightweight fetch → changelog diff.
metrics_service	api_gateway/services/metrics_service.py	get_metrics() - runs 3 DB queries (total triaged, recent completions, source list) plus a per-connector Redis loop to build per-severity /

		confidence / P0-P1 aggregate counts. Returns a single metrics dict the dashboard consumes.
<code>trriage_service</code>	<code>api_gateway/services/triage_service.py</code>	<code>start_triage(bug_id, source_id, user_id, force_refresh, kafka_producer)</code> - source detection, Kafka publish attempt, <code>asyncio.create_task</code> local pipeline fallback. <code>get_triage_result(case_id)</code> - Redis fast path → DB fallback → 404. <code>get_triage_history(limit)</code> - queries <code>audit_log</code> and formats entries.
<code>case_service</code>	<code>api_gateway/services/case_service.py</code>	<code>get_case_result(case_id)</code> - Redis case cache → stored panels fallback via <code>get_stored_panels</code> → direct Redis key reads for related:, kb:, enrichment: when panels are missing. <code>_context_from_panels</code> maps panel names to the context dict shape the frontend expects.
<code>connection_service</code>	<code>api_gateway/services/connection_service.py</code>	<code>create_connection(...)</code> - slugifies display name, mutates <code>os.environ</code> , health-checks via connector before DB write, writes to <code>source_registry</code> , invalidates connector cache. <code>list_connections()</code> - DB + <code>health_check_all</code> . <code>test_connection(source_id)</code> - instantiates connector, classifies Timeout vs Error.
<code>__init__</code>	<code>api_gateway/services/_init__.py</code>	Package initializer - no logic.

Group 4 - Orchestrator Core · `orchestrator/`

Module	File Location	Responsibility
<code>orchestrator</code>	<code>orchestrator/orchestrator.py</code>	<code>TaskOrchestrator.run(case_id, bug_id, source_id, engineer_id, force_refresh)</code> - full 4-agent pipeline (ContextFetch → CrossSystemFetch Enrichment → AISynthesis). Manages DB checkpoints, panel publishing, 1.5s WebSocket race-condition delay, pipeline-complete event, audit log write, related-group persistence, and bug-list cache invalidation.
<code>kafka_consumer</code>	<code>orchestrator/kafka_consumer.py</code>	Infinite retry loop around <code>AIOKafkaConsumer</code> (earliest offset, manual commit, 45s session timeout). On each message: calls <code>TaskOrchestrator().run(...)</code> , then commits offset. Sleeps 10s on <code>KafkaError</code> before reconnecting.
<code>redis_client</code>	<code>orchestrator/redis_client.py</code>	Singleton <code>aioredis.Redis</code> client. Cache functions: ticket cache, buglist cache, case result cache, purge. Panel functions: <code>store_panel_update</code> writes <code>panel:{case_id}:{name}</code> + appends to <code>panels:{case_id}</code> list. <code>publish_pipeline_done</code> fires completion event. <code>get_stored_panels</code> returns ordered replay for late-joining WebSocket clients.
<code>main</code>	<code>orchestrator/main.py</code>	CLI entry point - calls <code>consume_triage_requests()</code> inside <code>asyncio.run</code> . Used when running the orchestrator as a standalone process separate from the FastAPI app.
<code>__init__</code>	<code>orchestrator/_init__.py</code>	Package initializer - no logic.

Group 5 - Pipeline Agents · `orchestrator/agents/`

Module	File Location	Responsibility
<code>base</code>	<code>orchestrator/agents/base.py</code>	Abstract <code>BaseAgent</code> with <code>step_name</code> property, abstract <code>run(context)</code> method, and <code>safe_run</code> wrapper that catches exceptions into

		context["errors"] and returns (context, duration_ms). All agents must implement run(context).
context_fetch	orchestrator/agents/context_fetch.py	Phase 1. Resolves connector by source_id or ticket-prefix match. Fetches primary ticket, truncates oversized fields, fetches linked items, searches customer portals in parallel, extracts co-references from ticket text (JIRA PROJECT-NNNNN, GitHub #NNN). Populates primary_ticket, bug_context, customer_cases, source_references, components.
cross_system_fetch	orchestrator/agents/cross_system_fetch.py	Phase 2a. 3-level related-ticket search: L1 backlink extraction (regex), L2 Redis cache scan + Groq LLM scoring (llama-3.1-8b-instant), L3 live API search with platform-specific queries (Groq-generated or deterministic fallback). Normalises all candidates, deduplicates, sorts by similarity_score.
enrichment	orchestrator/agents/enrichment.py	Phase 2b (parallel with CrossSystem). ReAct loop over Confluence (max 4 iterations). Iteration 0 runs Confluence + Stack Overflow in parallel. After loop: fetches Apache JIRA and GitHub Issues in parallel. Tags each article by source, merges all, caps at 6. Populates kb_articles, enrichment_sources.
ai_synthesis	orchestrator/agents/ai_synthesis.py	Phase 3. Calls Groq (llama-3.3-70b-versatile) with 2 attempts + fast-model JSON repair fallback. Normalises confidence (0-10 to 0-1). Validates via SynthesisOutput Pydantic model. 1-hour Redis per-bug cache keyed on triage_result:{source_id}:{bug_id}. Resolves or mints BT-NNN group via _resolve_group_id.
__init__	orchestrator/agents/__init__.py	Re-exports all four agents and BaseAgent.

Group 6 - Connectors · orchestrator/connectors/

Module	File Location	Responsibility
base_connector	orchestrator/connectors/base_connector.py	Abstract BaseConnector: sets source_id, system_type, base_url, project_key, ticket_prefix, token. Properties: is_bug_source, is_knowledge_source, is_customer_source. Abstract methods: get, search, get_linked_items, get_lightweight, extract_links. Default health_check calls search_open_bugs(max_results=1) and returns latency + ok/error dict.
registry	orchestrator/connectors/registry.py	ConnectorRegistry: get_all_enabled always calls load_connectors_from_db (no in-process TTL cache). Methods: get(source_id), get_by_ticket_id (longest-prefix-first), get_by_type, health_check_all, invalidate_cache. SYSTEM_TYPE_TO_CLASS dict maps type strings to connector classes. EnvironmentTokenProvider reads tokens from os.environ.
github_connector	orchestrator/connectors/github_connector.py	GithubConnector: calls GitHub REST API v3. Implements get (/repos/{repo}/issues/{id}), search (issues or search API), get_lightweight (status + severity from labels), get_changelog (events API), extract_links (scans body for JIRA URLs, BZ URLs, #NNN refs). Maps GitHub labels to P0-P3 severity.
jira_connector	orchestrator/connectors/jira_connector.py	JiraConnector: calls Apache JIRA REST API v2. Implements get (expands changelog), search (JQL: project = X AND text ~ query), get_lightweight, get_changelog (values/histories), extract_links (issuelinks block, remote web links, description scan). Handles ADF description parsing for JIRA Cloud v3.

<code>bugzilla_connector</code>	<code>orchestrator/connectors/bugzilla_connector.py</code>	BugzillaConnector: calls Bugzilla REST API (Mozilla Firefox, Linux Kernel). Implements <code>get (/rest/bug/{id})</code> , <code>search</code> (open statuses or quicksearch), <code>get_lightweight</code> , <code>get_changelog (/rest/bug/{id}/history)</code> , <code>extract_links</code> (depends_on, blocks, see_also URLs). Maps Bugzilla status/priority to internal severity values.
<code>confluence_connector</code>	<code>orchestrator/connectors/confluence_connector.py</code>	ConfluenceConnector: calls Confluence REST API v1. <code>search</code> uses CQL (space = X AND text ~ query), strips HTML, builds canonical URL. Supports Atlassian Cloud (Basic Auth) and server (Bearer). Used by <code>EnrichmentAgent</code> . <code>get_linked_items</code> , <code>get_changelog</code> , <code>get_lightweight</code> , <code>extract_links</code> are all stubs returning empty.
<code>support_kb_connector</code>	<code>orchestrator/connectors/support_kb_connector.py</code>	SupportKBConnector: trivial subclass of <code>ConfluenceConnector</code> with no overrides. Inherits the full Confluence search contract. Used to represent HPE Engineering KB as a distinct source in the registry.
<code>customer_portal_connector</code>	<code>orchestrator/connectors/customer_portal_connector.py</code>	CustomerPortalConnector: searches Hacker News (hn.algolia.com) and Stack Overflow (stackexchange.com) in parallel. Scores signals by points/comments (HN) and views/score (SO). Returns <code>TicketData</code> objects with <code>signal_metadata</code> and <code>signal_score</code> for customer impact estimation. <code>get</code> , <code>get_linked_items</code> , <code>get_changelog</code> , <code>get_lightweight</code> , <code>extract_links</code> are all stubs.
<code>__init__</code>	<code>orchestrator/connectors/__init__.py</code>	Re-exports all connectors, <code>ConnectorRegistry</code> , and <code>get_connector_for_ticket</code> .

Group 7 - Database Layer · orchestrator/db/

Module	File Location	Responsibility
<code>base</code>	<code>orchestrator/db/base.py</code>	Base = <code>DeclarativeBase()</code> - SQLAlchemy ORM base class imported by all model classes. No logic beyond the declaration.
<code>session</code>	<code>orchestrator/db/session.py</code>	Creates <code>create_async_engine</code> from <code>POSTGRES_URL</code> . <code>AsyncSessionLocal</code> sessionmaker. <code>get_db</code> FastAPI dependency generator. <code>init_db</code> runs <code>Base.metadata.create_all</code> . <code>ensure_runtime_schema</code> applies idempotent ALTER TABLE migrations for <code>bug_group_mappings</code> columns.
<code>models</code>	<code>orchestrator/db/models.py</code>	All ORM table definitions: <code>UserRole</code> , <code>CustomerCase</code> , <code>SourceRegistry</code> (connector config), <code>PipelineContext</code> (resume state), <code>AuditLog</code> (per-triage record with JSONB summary), <code>CMDBTeamRegistry</code> , <code>SLAConfig</code> , <code>KBArticle</code> , <code>SystemGroupRegistry</code> (BT-NNN groups), <code>BugGroupMapping</code> (ticket-group many-to-many).
<code>source_registry</code>	<code>orchestrator/db/repositories/source_registry.py</code>	<code>get_all_sources</code> , <code>get_enabled_sources</code> , <code>get_source_by_id</code> , <code>create_source</code> , <code>update_source</code> (via SQLAlchemy update), <code>set_source_enabled</code> . CRUD over the <code>source_registry</code> table - the connector configuration store.
<code>audit_log</code>	<code>orchestrator/db/repositories/audit_log.py</code>	<code>insert_audit_entry</code> , <code>get_last_triage_for_bug</code> (most recent <code>pipeline_complete</code> for a <code>bug_id</code>), <code>get_last_triage_by_case_id</code> , <code>get_metrics_summary</code> (total count), <code>list_recent_pipeline_completions</code> .
<code>pipeline_context</code>	<code>orchestrator/db/repositories/pipeline_context.py</code>	<code>create_pipeline_context</code> , <code>get_pipeline_context</code> , <code>update_pipeline_step</code> , <code>delete_pipeline_context</code> . <code>get_steps_to_run(from_step)</code> returns the tail of <code>PIPELINE_STEPS</code> from the given resume point - used for crash recovery.

<code>user_roles</code>	orchestrator/db/repositories/user_roles.py	get_user_by_email, create_user (pbkdf2_sha256 via passlib), list_users, delete_user. Uses user_id = email as primary key in the user_roles table.
<code>group_registry</code>	orchestrator/db/repositories/group_registry.py	get_next_group_id (count-based BT-NNN), create_group, get_group_for_ticket, add_tickets_to_group, get_tickets_in_group, persist_related_issue_group (upsert-style: join existing group or mint new one, store role/similarity metadata per ticket).
<code>__init__</code>	orchestrator/db/__init__.py	Re-exports Base, engine, AsyncSessionLocal, get_db, models as a flat namespace.

Group 8 - Models & Schemas · orchestrator/models/ + orchestrator/utis/		
Module	File Location	Responsibility
<code>ticket</code>	orchestrator/models/ticket.py	TicketData dataclass: canonical ticket representation passed between connectors and agents (ticket_id, title, description, severity, status, component, assignee, timestamps, url, comments, linked_items, labels, direct_reference_links). ChangeEvent dataclass: field/old/new/changed_at/changed_by. Every connector's get and search must return these.
<code>synthesis</code>	orchestrator/models/synthesis.py	SynthesisOutput Pydantic model: unified_severity, status_summary, affected_components, root_cause, recommended_actions, engineer_summary, customer_summary, confidence (0.0-1.0 validated), reasoning, used_fallback. CandidateScore Pydantic model: ticket_id, similarity_score, label, reason, matching_fields.
<code>grouped_bug</code>	orchestrator/models/grouped_bug.py	GroupedBugRow Pydantic model: group_id, title, priority, status, primary_source_id, source_ids, system_types, ticket_ids, last_triaged_at, needs_retriage, changes_count. Represents one row in the grouped bug list view returned by GET /bugs.
<code>url_utils</code>	orchestrator/utis/url_utils.py	sanitize_bug_url(url, system_type, bug_id, base_url) - returns URL unchanged if real; if it contains a mock/placeholder domain and a base_url is provided, reconstructs a valid URL using system-type routing (JIRA /browse/, GitHub /issues/, Bugzilla /show_bug.cgi?id=).
<code>models/__init__</code>	orchestrator/models/__init__.py	Re-exports TicketData, ChangeEvent, SynthesisOutput, CandidateScore.

4.6 Communication Protocols

The system uses four communication protocols. Each was chosen for specific technical reasons.

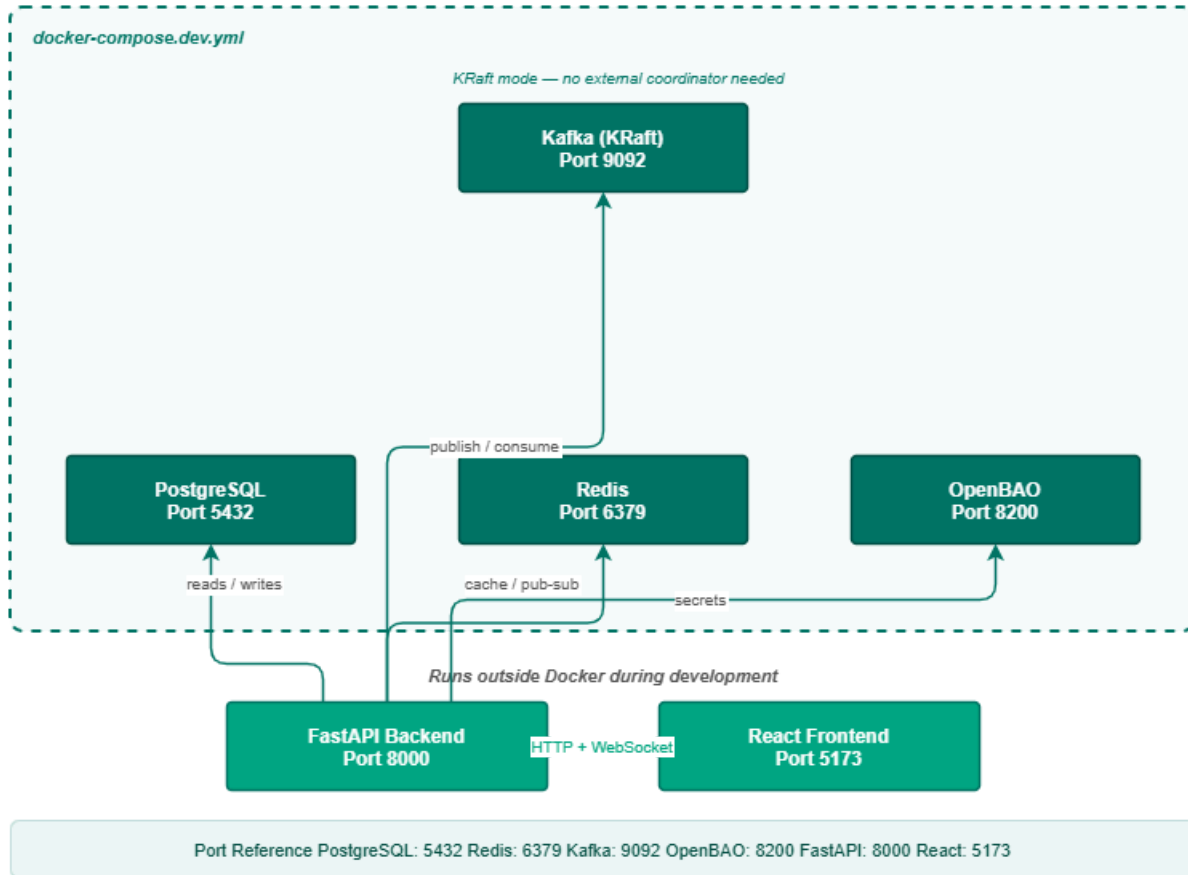
Protocol	Used Between	Why This Choice
HTTPS REST	Browser → FastAPI	All standard API calls: login, bug list, history, settings. FastAPI validates JWT and Pydantic models at the boundary before any handler logic runs.
WebSocket (ws://)	Browser ↔ FastAPI	Triage results stream panel-by-panel as agents finish. A standard HTTP request cannot push partial results incrementally. The JWT is passed as a query parameter since browser WebSocket APIs cannot set custom headers.
Apache Kafka	FastAPI → Orchestrator	Triage pipeline takes 8-10 seconds. A synchronous call would

		block the HTTP connection. Kafka also provides crash recovery: uncommitted offsets are replayed on orchestrator restart.
asyncpg (PostgreSQL wire protocol)	Orchestrator → PostgreSQL	All database operations use async parameterised queries. pipeline_context updates use explicit transactions ensuring a step either writes its full output or writes nothing.
aioredis (Redis RESP protocol)	All services → Redis	Cache reads/writes and pub/sub in a single async client. Connection pooling means no per-request connection overhead.
httpx (HTTPS)	Connectors → External APIs	All outbound calls to GitHub, JIRA, Bugzilla, Confluence use httpx with per-connector timeouts.

4.7 Infrastructure

The `docker-compose.dev.yml` file defines five services: PostgreSQL on port 5432, Redis on port 6379, Kafka on port 9092 running in KRaft mode with no Zookeeper dependency, OpenBAO on port 8200, and the FastAPI backend on port 8000. The React frontend runs outside Docker during development for faster hot-reload cycles. `ENABLE_LOCAL_PIPELINE_FALLBACK=true` runs the orchestrator as an asyncio background task inside FastAPI when Kafka is not available, which simplifies development without changing any application logic.

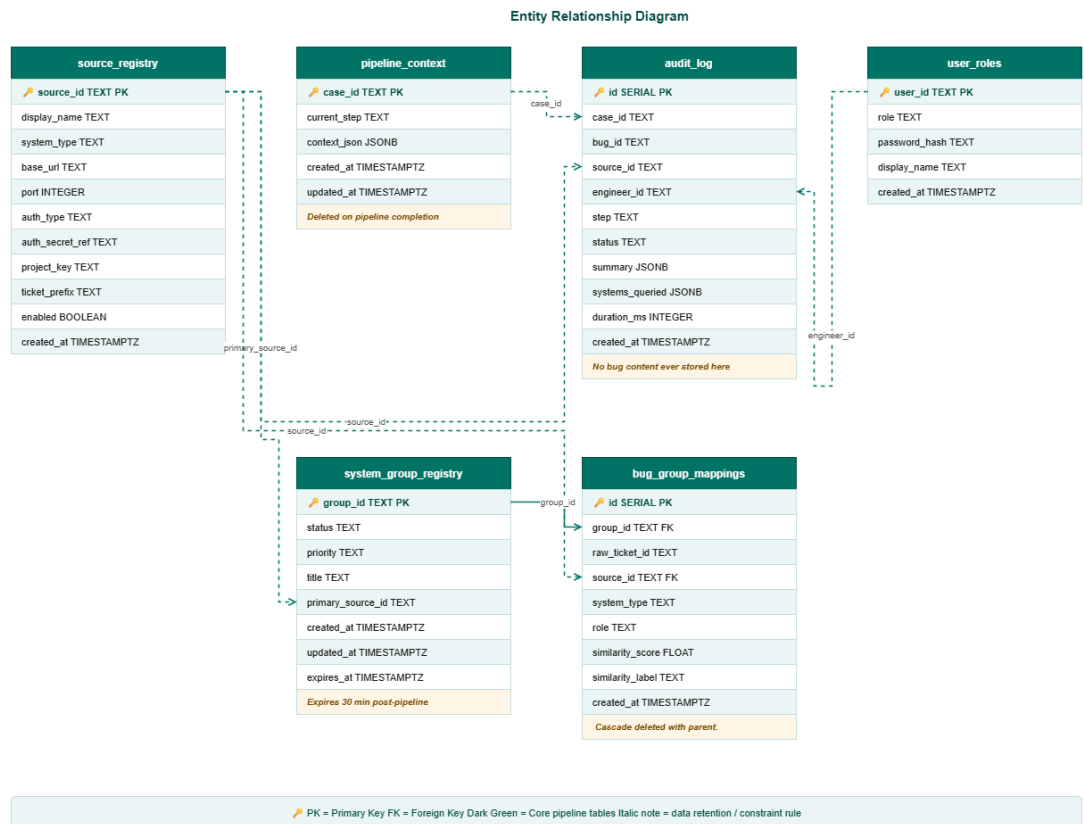
Docker Compose — Service Dependencies and Port Layout



[Figure 4 - Docker Compose service dependencies and port layout]

5. Database Design

5.1 Entity Relationship Diagram



[Figure 5 - ER Diagram: source_registry, pipeline_context, audit_log, user_roles, system_group_registry, bug_group_mappings]

5.2 Table Descriptions

Table	Purpose	Retention
source_registry	One row per connected external system. The single source of truth for which trackers are active and how to reach them.	Permanent
pipeline_context	Crash-safe working memory for one active triage run. Written after each agent phase completes. Allows pipeline to resume from last completed step on orchestrator restart. <i>Deleted on pipeline completion.</i>	Deleted on pipeline completion

audit_log	Permanent record of the AI's own pipeline outputs. Never contains any bug content from external systems - only derived metadata. <i>No bug content ever stored here.</i>	Permanent
user_roles	One row per authenticated engineer. Stores identity and role for JWT-based access control.	Permanent
system_group_registry	One row per bug group (BT-NNN). Tracks the aggregate status and priority of a group of related tickets across systems. The primary_source_id identifies which connector owns the canonical bug.	Temporary(30 min post-pipeline)
bug_group_mappings	Pure structural join table. Records which raw ticket IDs from which connectors belong to which group, along with their similarity metadata. Contains no remote-owned display fields. <i>Stores relationship metadata only.</i>	Temporary(30 min cascade on parent deletion)

5.3 What Is and Is Not Stored

The data sovereignty rule stated simply: if a piece of information came from an external system, it must not be stored permanently in this database. Only the AI's own outputs - things this system generated, not things it read - can be stored permanently.

Data	Stored?	Location	Duration
Bug title	Yes- Temporarily	PostgreSQL system_group_registry	30 min
Description, comments	No	Never written anywhere permanently	N/A
Error logs, stack traces	No	Never written anywhere permanently	N/A
Raw API responses from trackers	Yes- Temporarily	Redis: buglist:{source}:{status}:key	2 minutes
Pipeline working state between agents	Yes- Temporarily	PostgreSQL pipeline_context table	Deleted when pipeline finishes
AI-assigned severity (P0–P3)	Yes - permanently	PostgreSQL audit_log.summary JSONB	Permanent
Confidence score from AI synthesis	Yes - permanently	PostgreSQL audit_log.summary JSONB	Permanent
Systems queried during triage	Yes - permanently	PostgreSQL audit_log.systems_queried	Permanent
Time of triage and who	Yes -	PostgreSQL audit_log	Permanent

triggered it	permanently		
All four panel results	Yes- Temporarily	Redis case result key	2 min
API tokens for connected systems	Yes - encrypted	OpenBAO vault (or .env in development)	Until connection is removed

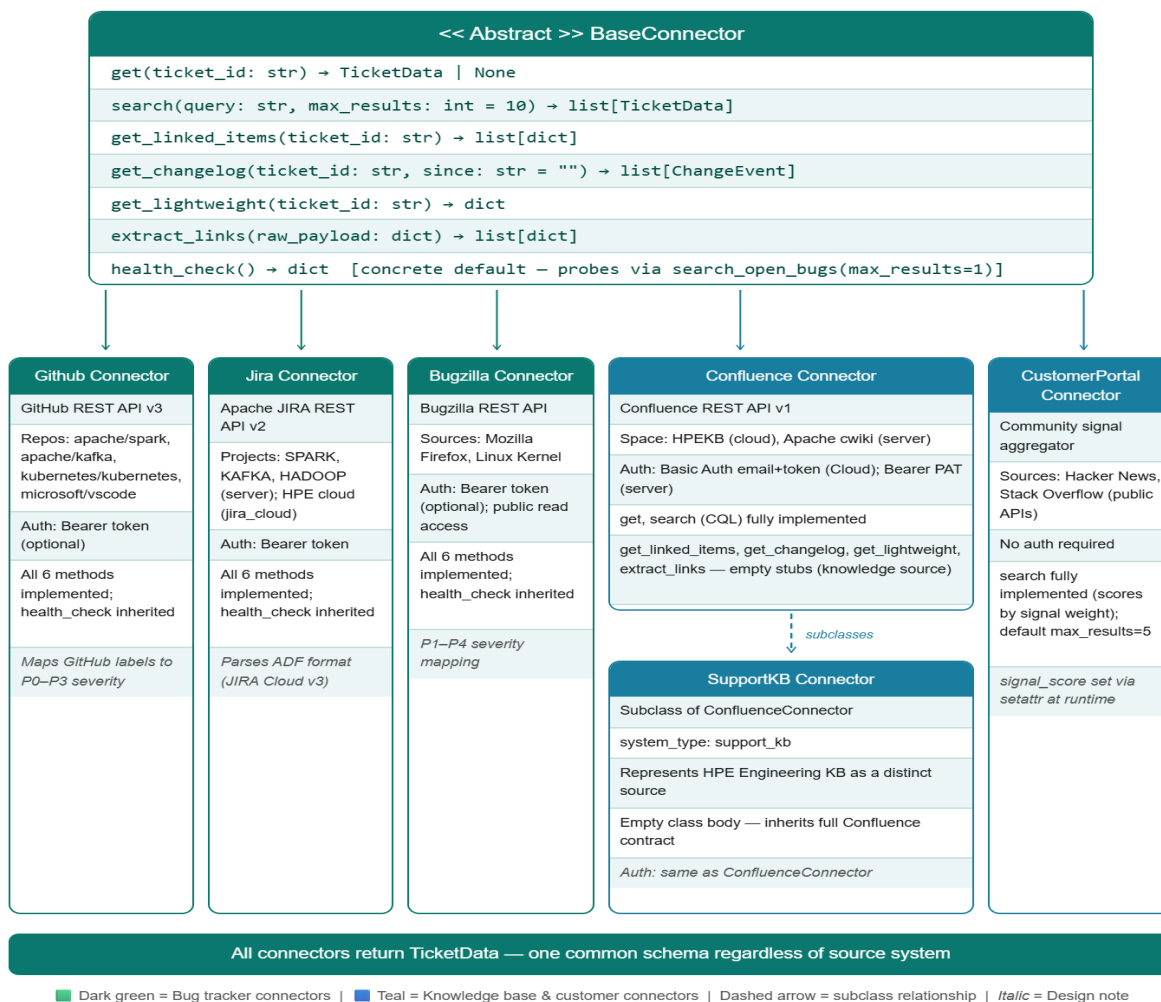
6. Connector Architecture

6.1 The Adapter Pattern

The Adapter Pattern is a structural design pattern that lets incompatible interfaces work together by inserting a translation layer between them. In this system, three external APIs return data in completely different formats. The pattern means each connector translates its system's format into TicketData, the common internal representation. The rest of the system only ever deals with TicketData objects and has no knowledge of whether data came from GitHub, JIRA, or Bugzilla.

The practical benefit is that adding a new bug tracker requires writing exactly one new Python class that implements the BaseConnector interface. The ConnectorRegistry discovers it automatically from the database. The pipeline uses it without modification. There are no system-type conditionals scattered through the codebase.

Adapter Pattern — BaseConnector Interface and Implementations



[Figure 6 - Adapter Pattern: BaseConnector interface and all connector implementations]

6.2 BaseConnector Interface

The following table maps each BaseConnector method to its implemented purpose and the point in the pipeline where it is invoked. The abstract class is defined in `orchestrator/connectors/base_connector.py`.

Method / Property	Implemented Purpose	When Called
<code>BUG_SOURCE_TYPES</code>	Defines supported bug sources: Jira, GitHub, Bugzilla, Jira Cloud, Jira Apache.	At registry load time - ConnectorRegistry uses this to classify connectors as bug sources.
<code>KNOWLEDGE_SOURCE_TYPES</code>	Defines documentation/support sources: Confluence and Support KB.	At registry load time - used to filter knowledge connectors for EnrichmentAgent.
<code>CUSTOMER_SOURCE_TYPES</code>	Defines customer-facing sources such as customer portals.	At registry load time - used to filter customer connectors for ContextFetchAgent.
<code>__init__()</code>	Initializes connector configuration from registry values like <code>source_id</code> , <code>system_type</code> , <code>base_url</code> , <code>project_key</code> , <code>ticket_prefix</code> , and <code>token</code> .	When <code>load_connectors_from_db()</code> instantiates each connector from the <code>source_registry</code> table.
<code>is_bug_source</code>	Checks whether the connector is a bug source.	In <code>ConnectorRegistry.get_by_ticket_id()</code> and bug list fetching to filter relevant connectors.
<code>is_knowledge_source</code>	Checks whether the connector is a knowledge/support source.	In <code>EnrichmentAgent</code> when selecting which connectors to query for KB articles.
<code>is_customer_source</code>	Checks whether the connector is a customer source.	In <code>ContextFetchAgent</code> when running parallel customer signal searches.
<code>get(ticket_id)</code>	Fetches full ticket details from the external system.	Phase 1 - <code>ContextFetchAgent</code> calls this to retrieve the primary ticket with all fields.
<code>get_ticket(ticket_id)</code>	Wrapper around <code>get()</code> to keep agent usage consistent.	Called by agents that prefer the <code>get_ticket()</code> alias over <code>get()</code> directly.
<code>search(query, max_results)</code>	Searches tickets/issues for matching results.	Phase 2a - <code>CrossSystemFetchAgent</code> (L3 fallback). Also called by background bug list ingestion loop in <code>main.py</code> .
<code>search_open_bugs()</code>	Fetches open bugs for the Live Bug List. Falls back to <code>search("")</code> .	Called by the background ingestion loop in <code>main.py</code> to populate Redis bug list cache.
<code>get_linked_items(ticket_id)</code>	Fetches directly linked/cross-referenced bugs for related issue discovery.	Phase 2a - <code>CrossSystemFetchAgent</code> L1 strategy for explicit backlink extraction.
<code>get_changelog(ticket_id, since)</code>	Optional changelog fetch for hybrid incremental change detection. Default returns empty list.	Called ONLY when Step 3 of change detection detects a field difference in <code>get_lightweight()</code> .
<code>get_lightweight(ticket_id)</code>	Fetches only lightweight metadata: <code>updated_at</code> , <code>severity</code> , and <code>status</code> .	Incremental change detection Step 2 - called periodically to check if a bug changed since last triage.

<code>lightweight_status(ticket_id)</code>	Wrapper around <code>get_lightweight()</code> for consistent status checking.	Called from <code>bug_service.get_bug_status()</code> to check current ticket state without a full fetch.
<code>health_check()</code>	Tests connector availability and latency using <code>search_open_bugs(max_results=1)</code> .	Called by <code>ConnectorRegistry.health_check_all()</code> - triggered on GET <code>/settings/connections</code> and POST <code>/settings/connections/{id}/test</code> .
<code>extract_links(raw_payload)</code>	Extracts explicit outbound references from raw connector payloads.	Phase 2a - <code>CrossSystemFetchAgent</code> L1 strategy, called immediately after <code>get()</code> to parse cross-system links from the raw response.

6.3 TicketData - Universal Bug Representation

`TicketData` is the common Python dataclass that all connectors normalise to. Defined in `orchestrator/models/ticket.py`. The pipeline never processes raw API responses from external systems - all connector-specific field mapping is contained within each connector's `_normalise()` method.

```
@dataclass
class TicketData:
    ticket_id: str
    title: str
    description: str
    severity: str
    status: str
    component: str
    assignee: str
    reporter: str
    created_at: str
    updated_at: str
    source_id: str
    system_type: str
    url: str = ''
    api_url: str = ''
    error_excerpt: str = '' # stack trace or crash summary
    steps_to_reproduce: str = ''
    customer_impact: str = ''
    comments: list = field(default_factory=list)
    linked_items: list = field(default_factory=list)
    labels: list = field(default_factory=list)
    direct_reference_links: list = field(default_factory=list)
```

6.4 Example Connector Implementations

source_id	Class	Type	Project	Auth
apache-spark-github	GithubConnector	GitHub Issues	apache/spark	PAT optional
apache-kafka-github	GithubConnector	GitHub Issues	apache/kafka	PAT optional
apache-flink-github	GithubConnector	GitHub Issues	apache/flink	PAT optional
kubernetes-github	GithubConnector	GitHub Issues	kubernetes/kubernetes	PAT optional

vscode-github	GithubConnector	Git Hub Issues	microsoft/vscode	PAT optional
apache-spark-jira	JiraConnector	Apache JIRA v2	SPARK	None - public
apache-kafka-jira	JiraConnector	Apache JIRA v2	KAFKA	None - public
apache-hadoop-jira	JiraConnector	Apache JIRA v2	HADOOP	None - public
apache-hive-jira	JiraConnector	Apache JIRA v2	HIVE	None - public
apache-flink-jira	JiraConnector	Apache JIRA v2	FLINK	None - public
mozilla-firefox-bugzilla	BugzillaConnector	Bugzilla REST	Firefox	None - public
hpe-confluence	ConfluenceConnector	Confluence Cloud	HPEKB	Basic Auth
hpe-customer-portal	CustomerPortalConnector	Internal Mock	-	None

A note on Atlassian Document Format: Atlassian Cloud JIRA returns the description field as ADF - a deeply nested JSON structure. JiraConnector includes a recursive `_extract_text_from_adf()` method that walks the ADF tree and produces clean readable text before any further processing. Without this, the text reaching the LLM would be unreadable JSON.

6.5 ConnectorRegistry

ConnectorRegistry is a module-level singleton that reads all enabled rows from `source_registry` on first use and caches instantiated connector objects. The cache invalidates when a connection is added or removed, making the new connector available on the very next request without restarting the server. The cache reloads automatically if found empty, preventing a silent failure mode where the first load fails and the system gets stuck with no connectors indefinitely.

6.6 Severity Normalisation

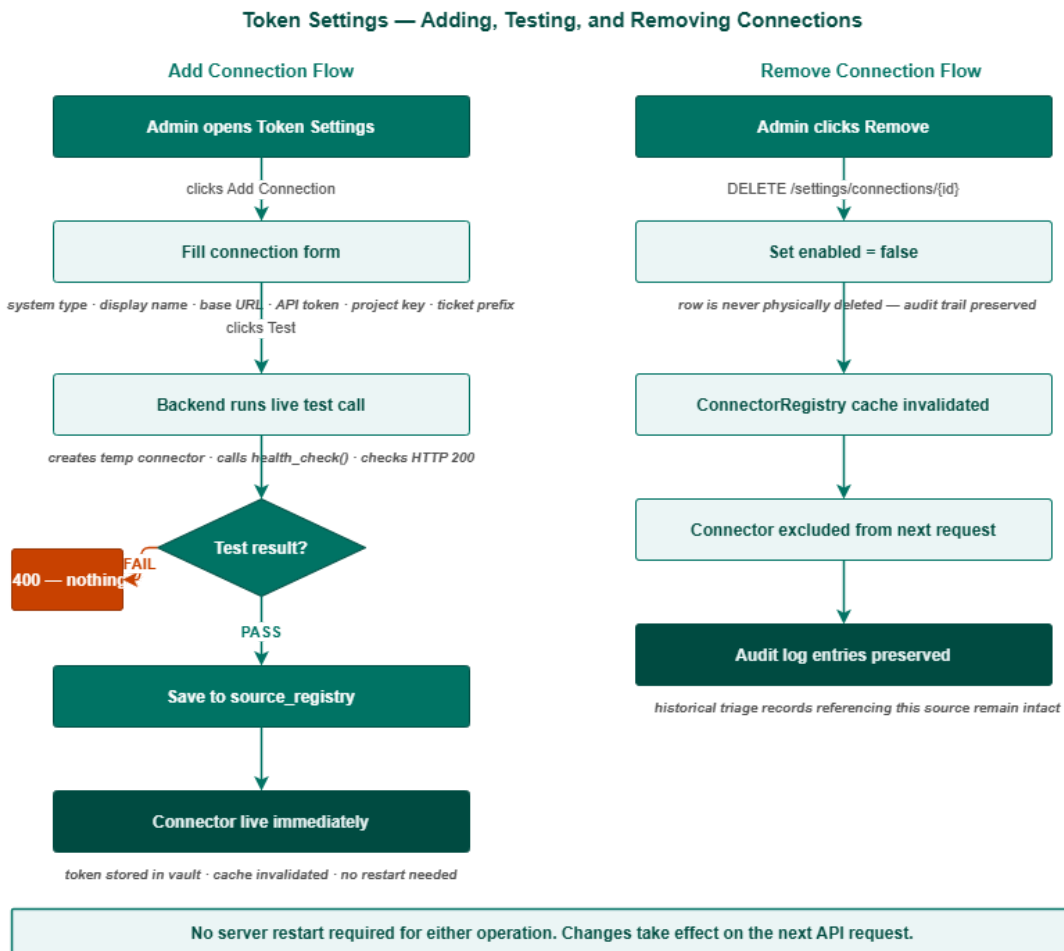
Internal Tier	GitHub Labels	JIRA Priority	Bugzilla Priority
P0 - Critical	priority:blocker, priority:critical	Blocker, Critical	P1, blocker, critical
P1 - High	priority:high	Major, High	P2, major
P2 - Medium	priority:medium, bug	Minor, Medium, Normal	P3, normal
P3 - Low	priority:low, enhancement	Trivial, Low	P4, enhancement

Priority Normalisation Strategy: The table above covers standard priority values for JIRA, GitHub, and Bugzilla. All listed values are resolved with a direct dictionary lookup - no LLM call, fully deterministic. When a connector returns a value not in the table - for example a custom JIRA priority like "Show Stopper" or "SEV-1" - the system falls back to an LLM call at `temperature=0`. The LLM receives the raw priority string, ticket title, and error excerpt and classifies it into P0–P3 using fixed definitions: P0 for production down or data loss, P1 for major feature broken, P2 for significant issue with a workaround, and P3 for minor or low-urgency issues.

When a new source system is added, its priority values are manually added to the existing mapping dictionary in the normalisation code, extending deterministic coverage without any structural changes to the pipeline.

6.7 Dynamic Configuration via Token Settings

The Token Settings page is the UI for managing source_registry. An engineer fills a form with system type, display name, base URL, optional token, project key, and ticket prefix. Before saving, the backend creates a temporary connector instance and makes a live test call. If the test succeeds, the row is inserted and the cache is invalidated. If it fails, 400 is returned and nothing is saved. The new source appears in the bug list on the next page load.



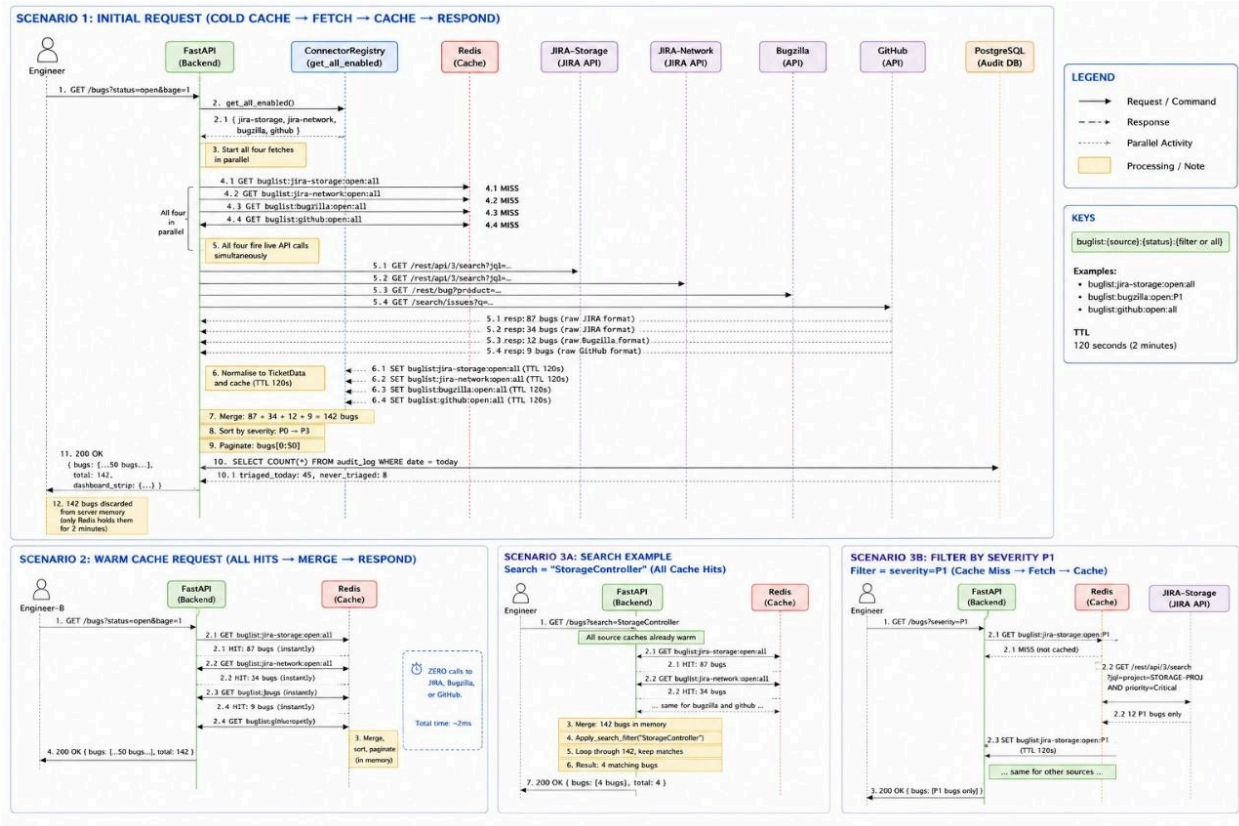
[Figure 7 - Token Settings page: adding, testing, and removing connections]

7. Sequence Diagrams

The ten sequence diagrams below document every primary interaction flow in the system. Each diagram shows the exact sequence of calls, data transfers, and decisions between components. Placeholders are provided for the team to insert the final drawn illustrations.

SD 1: Live Bug List

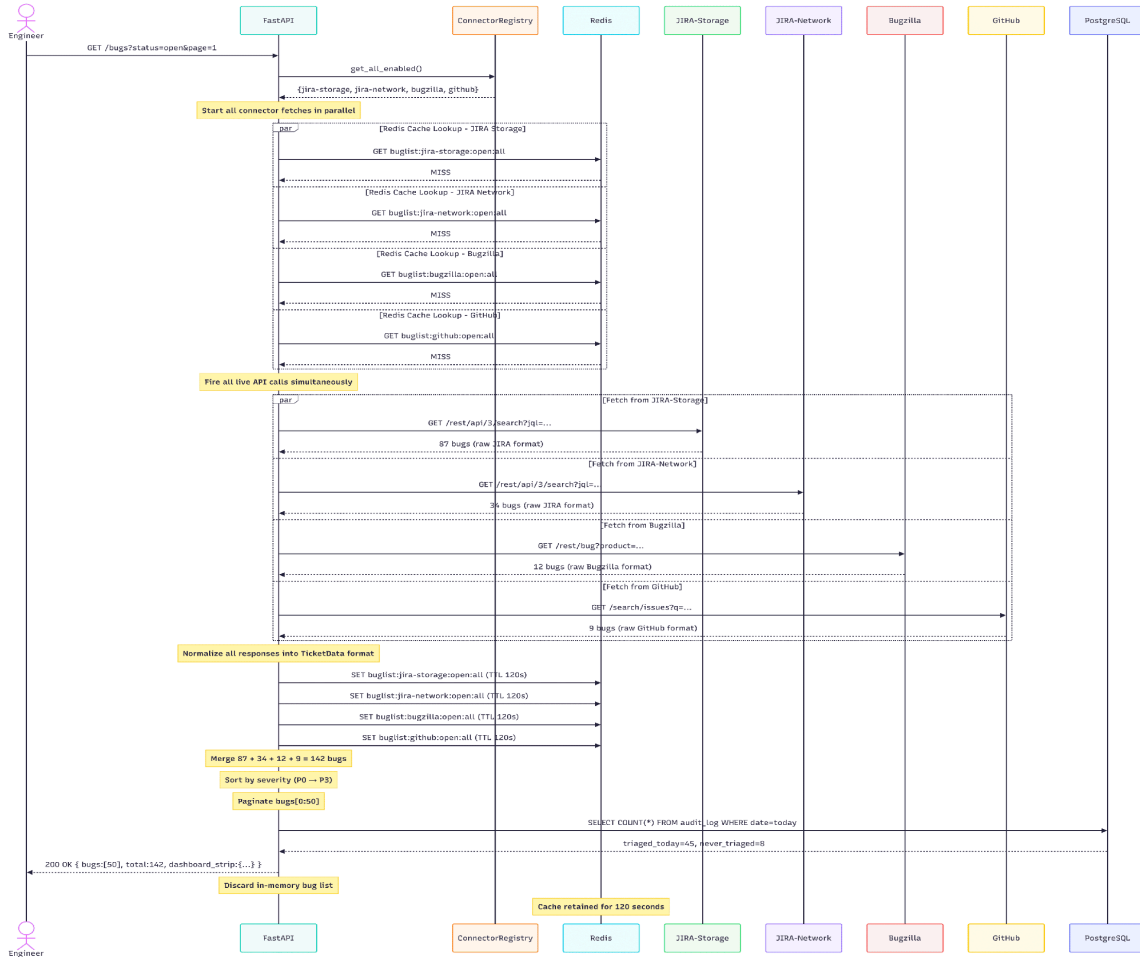
Actors: Engineer Browser, FastAPI, ConnectorRegistry, Redis, JIRA, Bugzilla, GitHub, PostgreSQL audit_log



SD 1 covers four scenarios that the engineer encounters when loading or filtering the bug list.

SD 1.1: Initial Page Load - Cold Cache Miss

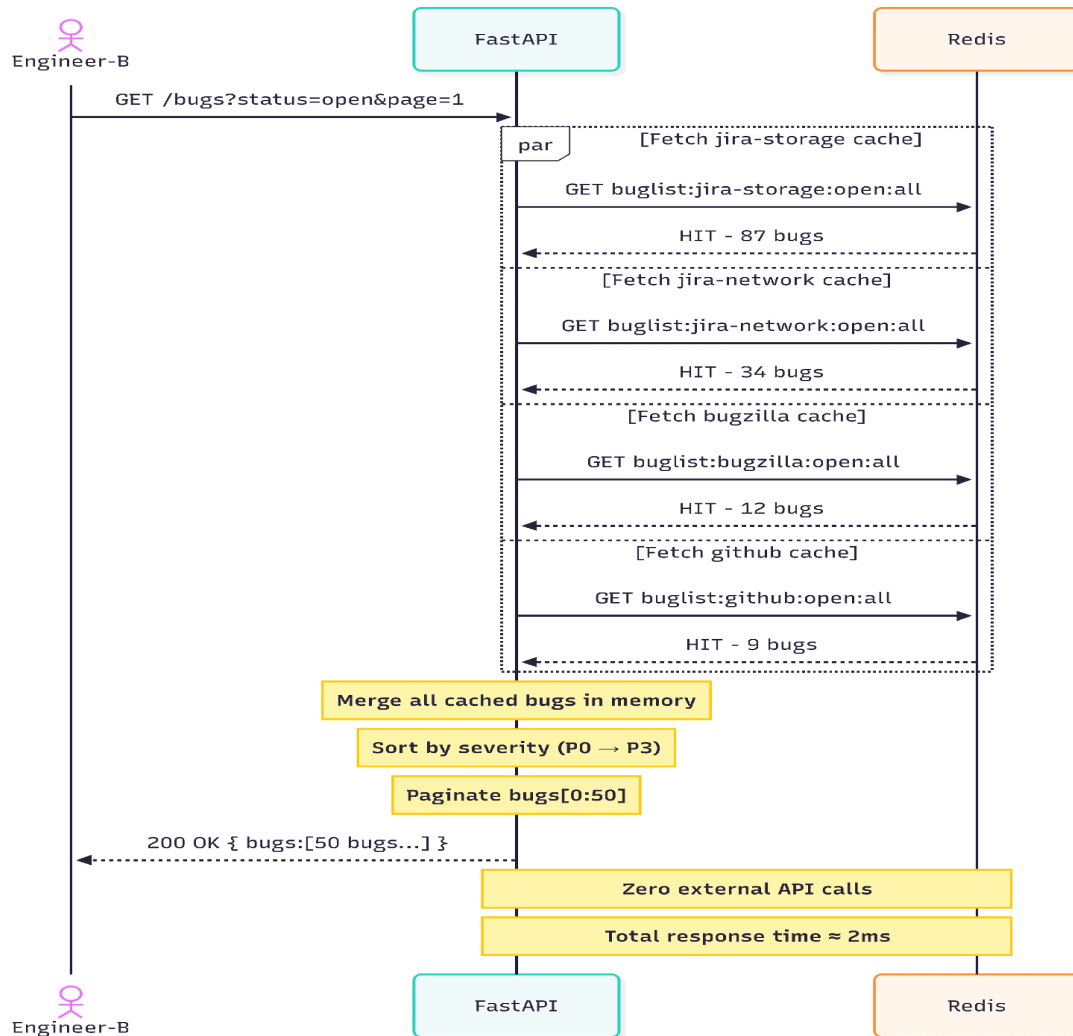
Actors: Engineer, FastAPI, ConnectorRegistry, Redis, JIRA-Storage, JIRA-Network, Bugzilla, GitHub, PostgreSQL



1. Engineer sends **GET /bugs?status=open&page=1**.
2. FastAPI calls **ConnectorRegistry.get_all_enabled()** → returns all 4 source connectors.
3. FastAPI starts all connector fetches in parallel using **asyncio.gather()**.
4. Each connector checks its Redis key first: **GET buglist:{source}:open**.
5. All 4 keys **MISS** (first load, cache is empty).
6. All 4 live API calls fire simultaneously: **JIRA-Storage, JIRA-Network, Bugzilla, and GitHub**.
7. Raw responses return from each system.
8. Each connector normalises its response into **TicketData** format.
9. Normalised results are written to Redis with a **120-second TTL**.
10. FastAPI merges all bugs in memory, sorts them by severity (**P0 → P3**), and paginates to **bugs[0:10]**.
11. FastAPI queries **audit_log** for **triaged_today** and **never_triaged** counts.
12. The system returns **200 OK** with the merged, sorted, and paginated bug list.

SD 1.2: Warm Cache Retrieval - All Cache Hits

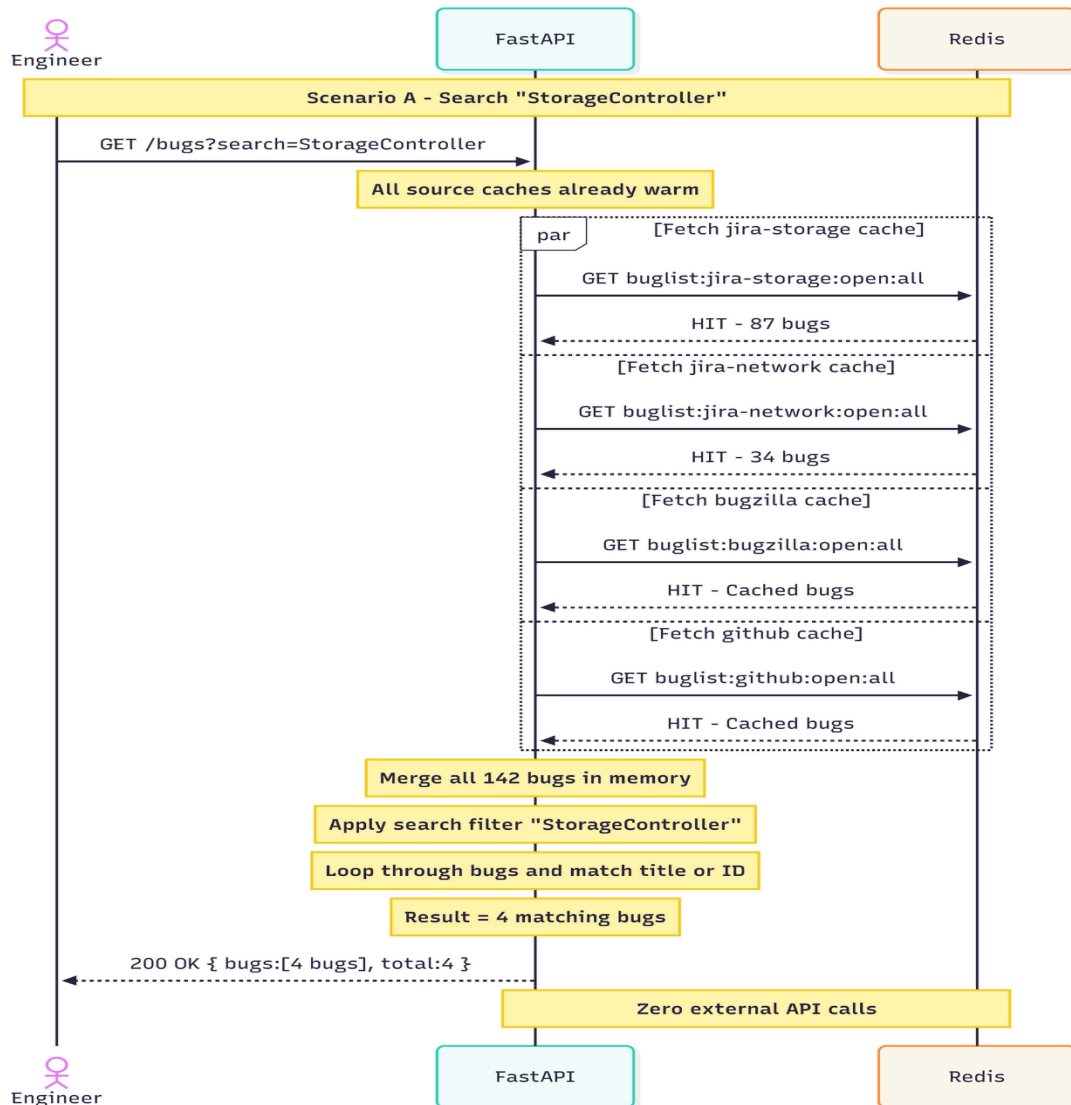
Actors: Engineer, FastAPI, Redis



1. Engineer sends GET /bugs?status=open&page=1 (within 120-second TTL window)
2. FastAPI checks all 4 Redis keys: GET buglist:{source}:open:all
3. All 4 keys HIT - no external API calls made
4. FastAPI reads all 4 cached lists from Redis in parallel
5. Merges and sorts all bugs in memory (P0 → P3)
6. Returns response in ~2ms

SD 1.3: Search Using Warm Cache

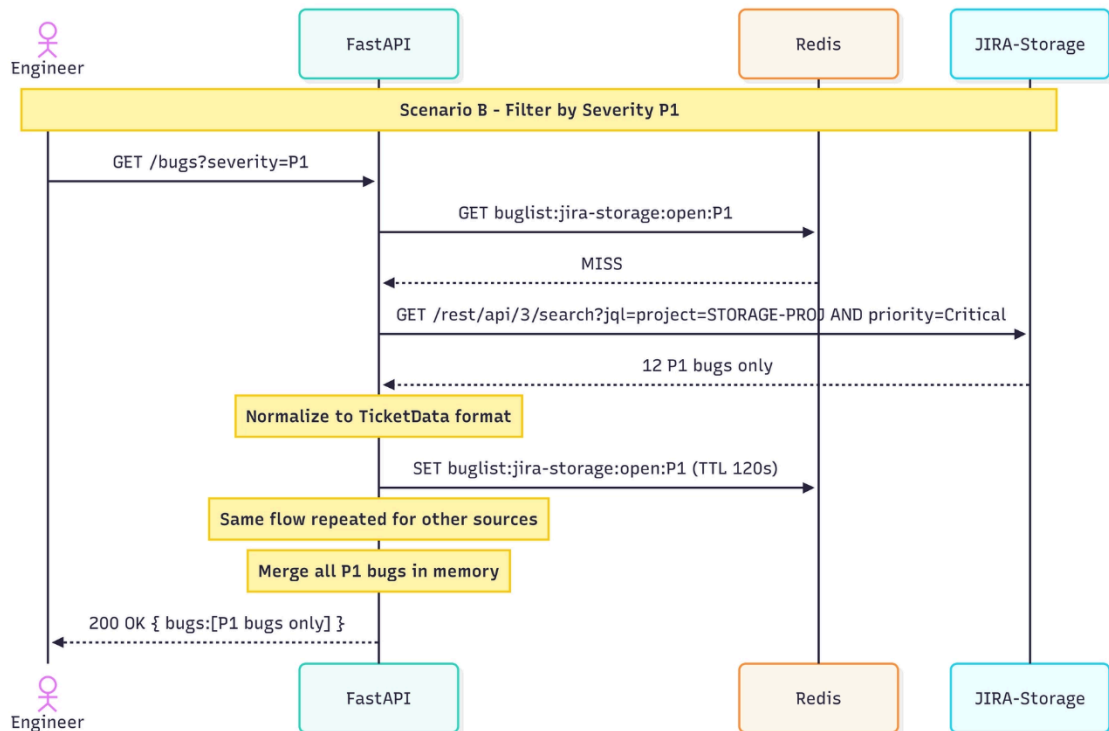
Actors: Engineer, FastAPI, Redis



1. Engineer sends GET /bugs?search=StorageController
2. FastAPI checks all 4 Redis keys → All 4 HIT (cache is warm)
3. FastAPI reads all 4 cached lists from Redis in parallel
5. 142 bugs loaded into memory from cache
6. FastAPI applies search filter: loops through merged list, matches on title or ticket_id
7. Returns 4 matching bugs

SD 1.4: Severity Filter - Partial Cache Miss

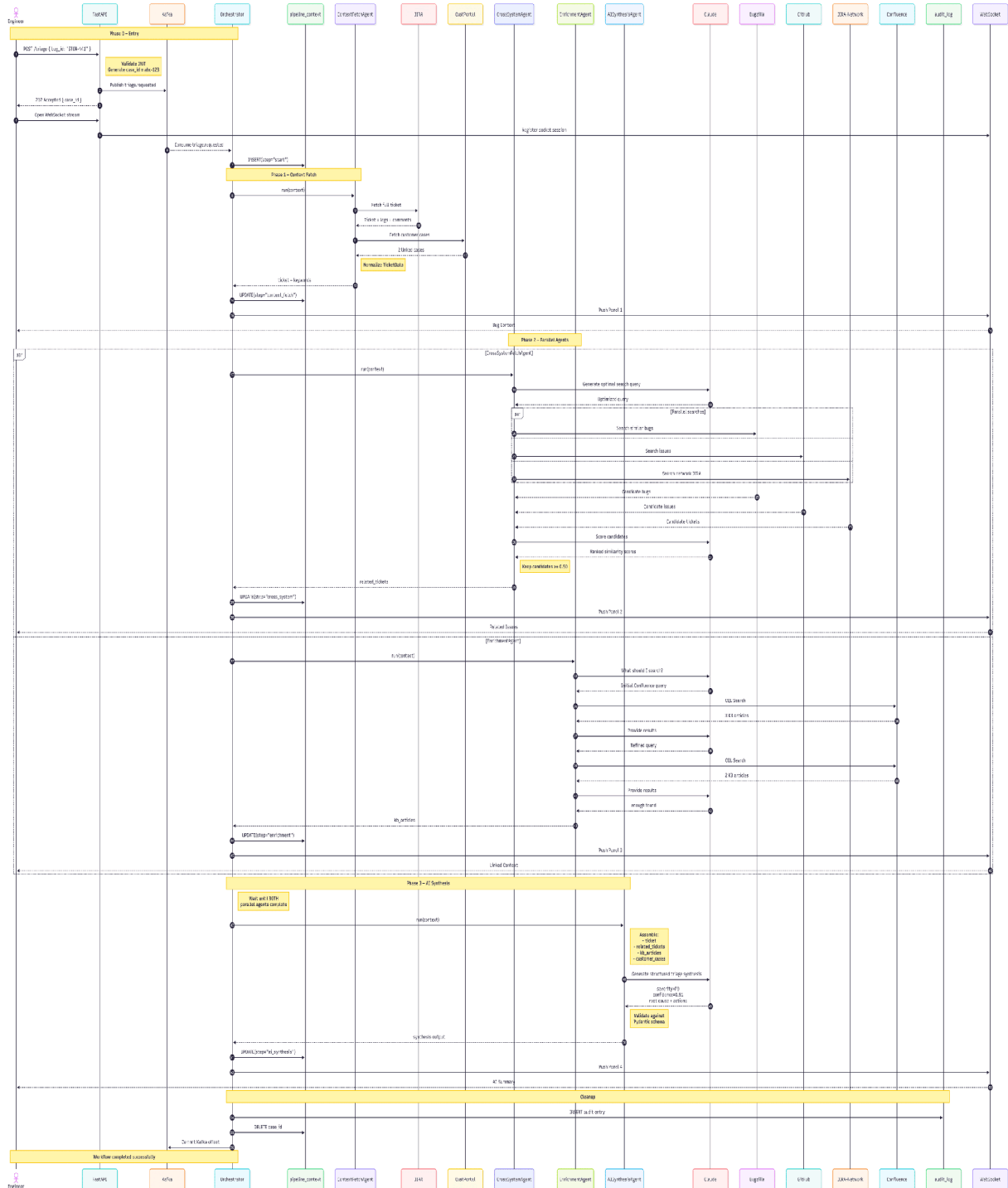
Actors: Engineer, FastAPI, Redis, JIRA-Storage



1. Engineer sends GET /bugs?severity=P1
2. FastAPI checks Redis key: GET buglist:jira-storage:open:P1
3. MISS - severity-filtered results are cached under a separate key from the all-severities key
4. FastAPI fires live query to JIRA: GET /rest/api/3/search?jql=project=STORAGE-PROJ AND priority=Critical
5. JIRA returns 12 P1 bugs
6. Normalised and cached as buglist:jira-storage:open:P1 with 120-second TTL
7. Same cache-check → miss → live-call → cache-write flow repeated for other sources
8. All P1 bugs merged and returned

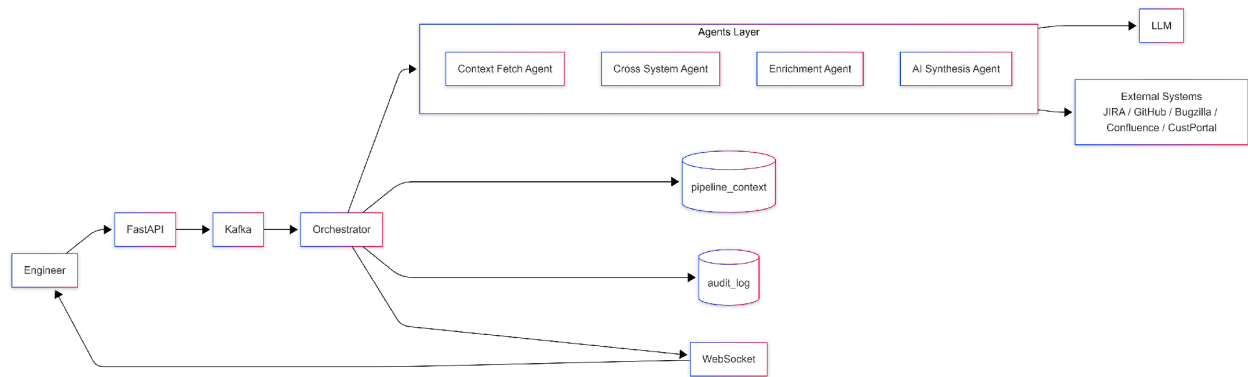
SD 2: Complete Triage Pipeline

Actors: Engineer, FastAPI, Kafka, Orchestrator, pipeline_context, ContextFetchAgent, CrossSystemFetchAgent, EnrichmentAgent, AISynthesisAgent, LLM, JIRA, CustPortal, Bugzilla, GitHub, Confluence, WebSocket, audit_log



SD 2 shows the complete end-to-end triage pipeline across five sub-diagrams. Each sub-diagram covers one phase of the pipeline.

SD 2.1: System Architecture Overview

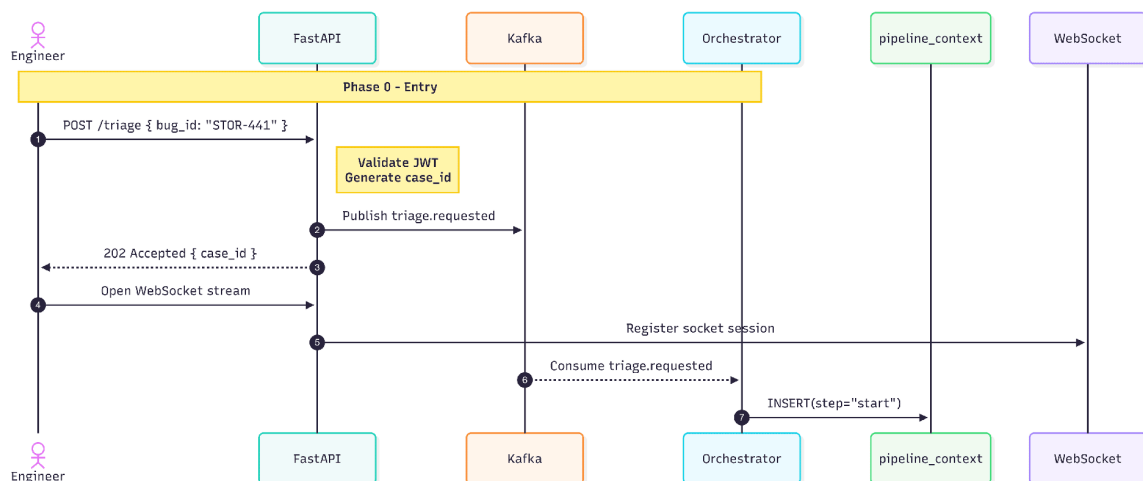


Engineer clicks Triage

- FastAPI (validates JWT, generates case_id)
- Kafka (publishes triage.requested event)
- Orchestrator (consumes event, manages phases)
- 4 Agents (ContextFetch → CrossSystemFetch + Enrichment → AISynthesis)
- LLM via Groq API + External Systems (JIRA, GitHub, Bugzilla, Confluence)
- PostgreSQL (pipeline_context checkpoints + audit_log write)
- Redis pub/sub → WebSocket → Engineer's browser (panel by panel)

SD 2.2: Request Entry & Event Trigger (Phase 0)

Actors: Engineer, FastAPI, Kafka, Orchestrator, pipeline_context, WebSocket

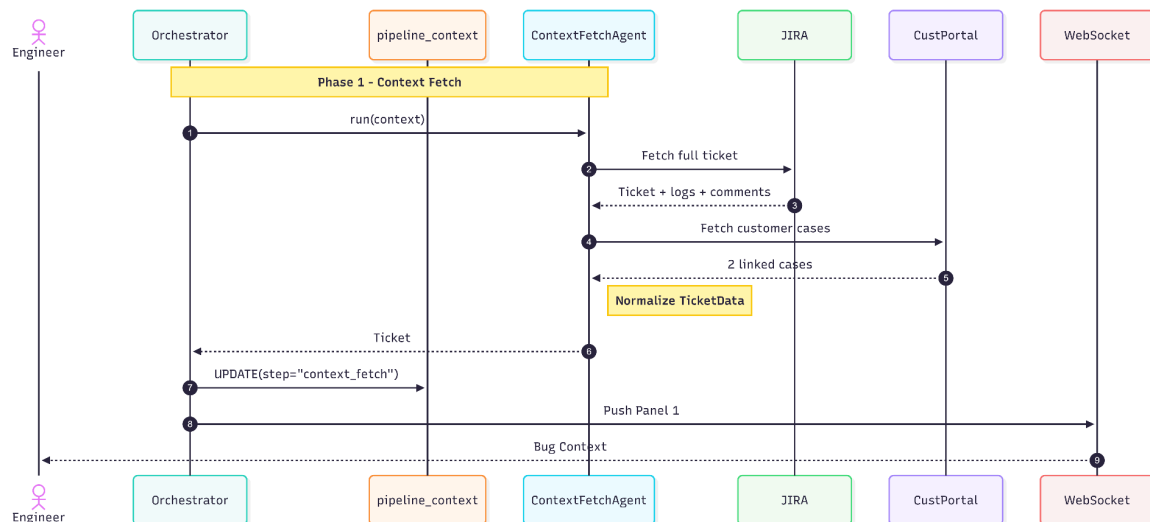


1. Engineer sends POST /triage { bug_id: "STOR-441" }

2. FastAPI validates JWT
3. FastAPI generates a unique case_id (UUID)
4. FastAPI publishes triage.requested event to Kafka topic triage.requests
5. FastAPI returns 202 Accepted { case_id } immediately (does not wait for pipeline)
6. Engineer's browser opens WebSocket: WS /triage/{case_id}/stream
7. FastAPI registers the WebSocket session and subscribes it to the Redis pub/sub channel for this case_id
8. Orchestrator consumes the triage.requested event from Kafka
9. Orchestrator inserts a pipeline_context row: { case_id, current_step: 'start', context_json: { bug_id } }

SD 2.3: Context Fetch Pipeline (Phase 1)

Actors: Orchestrator, pipeline_context, ContextFetchAgent, JIRA, CustPortal, WebSocket, Engineer

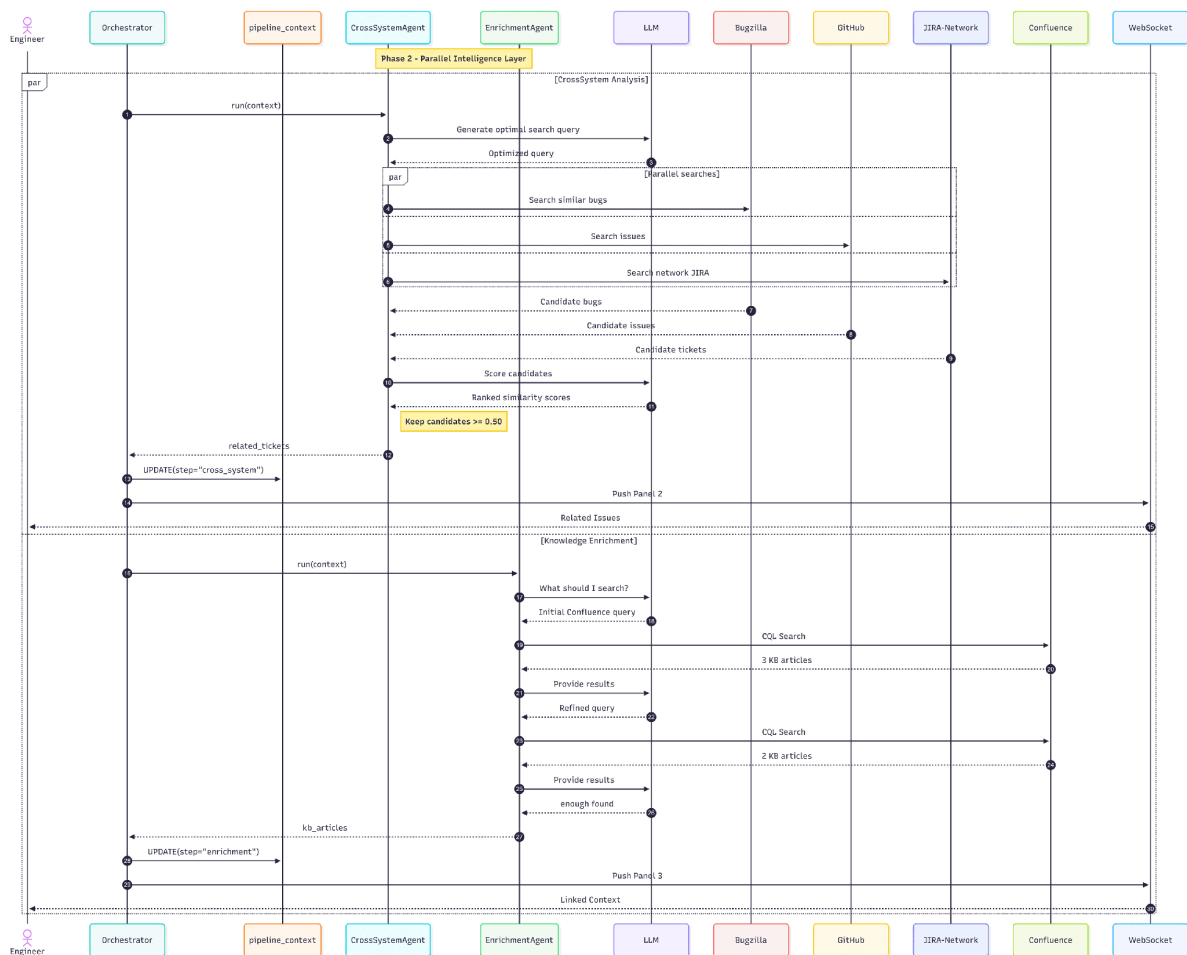


1. Orchestrator calls ContextFetchAgent.run(context)
2. ContextFetchAgent sends full ticket fetch to JIRA: GET /rest/api/2/issue/STOR-441
3. JIRA returns full ticket: title, description (ADF format), comments, error logs, linked issues
4. ContextFetchAgent parses ADF description using _extract_text_from_adf() → produces clean readable text
5. ContextFetchAgent fetches linked customer cases from Customer Portal using the ticket_id
6. Customer Portal returns 2 linked customer cases

7. ContextFetchAgent normalises all data into TicketData format
8. Returns { ticket: TicketData, customer_cases: [...] } to Orchestrator
9. Orchestrator updates pipeline_context: current_step = 'context_fetch', stores full context JSON
10. Orchestrator publishes Panel 1 (Bug Context) to Redis pub/sub channel
11. Panel 1 is also stored as a persistent Redis key (for late WebSocket replay)
12. Engineer's browser receives and renders Panel 1 - Bug Context

SD 2.4: Parallel Intelligence Layer (Phase 2)

Actors: Orchestrator, pipeline_context, CrossSystemFetchAgent, EnrichmentAgent, LLM, Bugzilla, GitHub, JIRA-Network, Confluence, WebSocket, Engineer



CrossSystemFetchAgent (Phase 2a):

1. Receives context containing the full primary TicketData
2. Calls LLM (Groq) with the primary ticket to generate a semantic search query (LLM understands the technical concept, not just keywords)
3. Fires the generated query in parallel against all other connected systems:
 - BugzillaConnector.search(query)
 - GithubConnector.search(query)
 - JiraConnector.search(query)

Collects all candidate tickets from all systems

4. Calls LLM again to score each candidate against the primary ticket (0.0 → 1.0) Scoring weights: error signatures > steps to reproduce > occurrence context > title similarity
5. Filters: keeps only candidates with score ≥ 0.50
6. Returns related_tickets list to Orchestrator
7. Orchestrator updates pipeline_context: current_step = 'cross_system'
8. Orchestrator publishes Panel 2 (Related Issues) to Redis pub/sub + persistent key
9. Engineer's browser receives and renders Panel 2 - Related Issues

EnrichmentAgent (Phase 2b - runs simultaneously with 2a):

1. Receives context containing primary TicketData and keywords
2. Round 1: Calls LLM asking "What should I search for in Confluence?" LLM reasons and emits an initial CQL query
3. EnrichmentAgent executes CQL search against Confluence: GET /wiki/rest/api/content/search?qql=...
4. Returns 3 articles as observation back to LLM
5. Round 2: LLM reads the 3 articles, reasons they are too general, refines the CQL query
6. EnrichmentAgent executes second CQL search
7. Returns 2 more specific articles as observation
8. Round 3 (if needed): LLM evaluates results. If satisfied → emits Final Answer
9. EnrichmentAgent returns kb_articles list to Orchestrator (If all 3 rounds fail to find articles → falls back to local kb_articles PostgreSQL table)

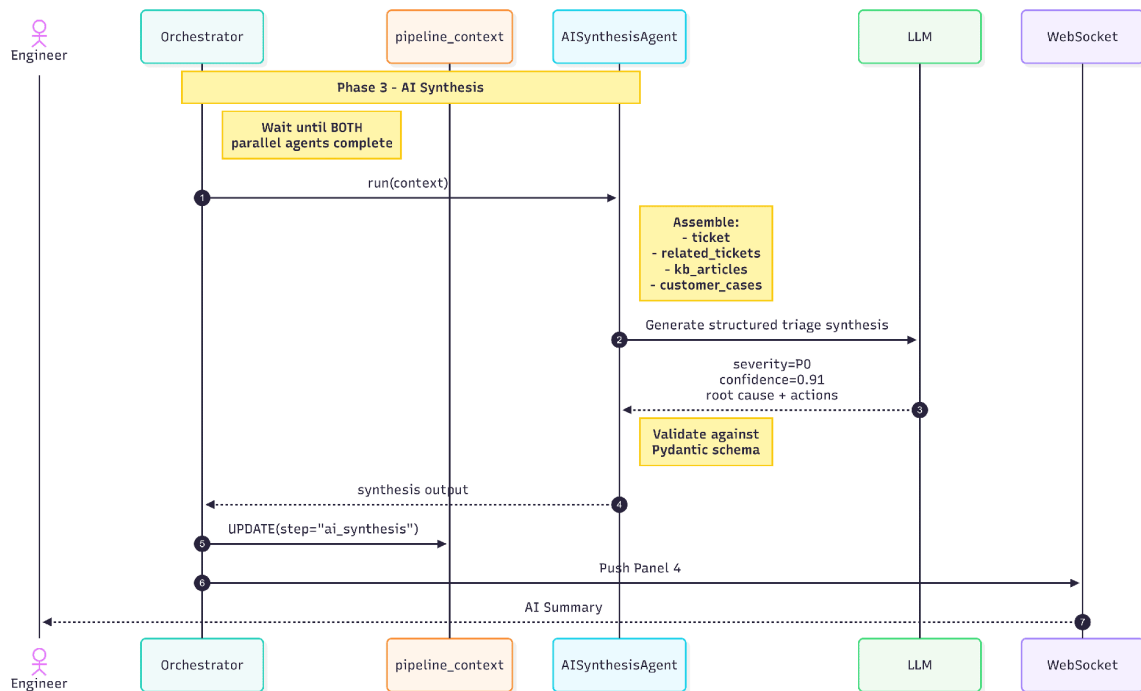
10.Orchestrator updates pipeline_context: current_step = 'enrichment'

11.Orchestrator publishes Panel 3 (Linked Context) to Redis pub/sub + persistent key

12.Engineer's browser receives and renders Panel 3 - Linked Context

SD 2.5: AI Synthesis & Structured Output (Phase 3)

Actors: Orchestrator, pipeline_context, AISynthesisAgent, LLM, WebSocket, Engineer



1. Orchestrator waits for both Phase 2 agents to finish (asyncio.gather() resolves)

2. Orchestrator calls AISynthesisAgent.run(context)

3. AISynthesisAgent assembles the full context package:

- Primary ticket (TicketData from Phase 1)
- Related tickets (related_tickets from Phase 2a)
- KB articles (kb_articles from Phase 2b)
- Customer cases (from Phase 1)

4. Sends the full assembled context to Groq LLM with temperature=0

5. LLM returns structured JSON:

json

{

```

"severity": "P0",

"confidence": 0.91,

"root_cause": "...",

"recommended_actions": ["...", "..."],

"affected_components": ["StorageController"],

"engineer_summary": "..."

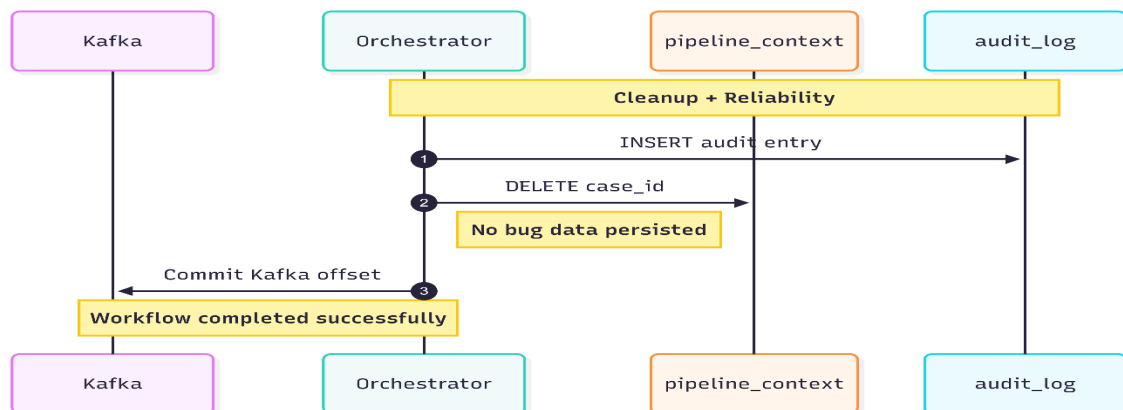
}

```

6. AISynthesisAgent validates response against Pydantic SynthesisOutput schema
7. If validation fails → retry with explicit JSON schema in prompt
8. If retry fails → deterministic keyword classifier as final fallback
9. Returns validated SynthesisOutput to Orchestrator
10. Orchestrator updates pipeline_context: current_step = 'ai_synthesis'
11. Orchestrator publishes Panel 4 (AI Summary) to Redis pub/sub + persistent key
12. Engineer's browser receives and renders Panel 4 - AI Summary

SD 2.6: Cleanup & Reliability

Actors: Kafka, Orchestrator, pipeline_context, audit_log



1. INSERT audit entry into audit_log:

```
{ bug_id, engineer_id, step, summary (JSONB), systems_queried, duration_ms }
```

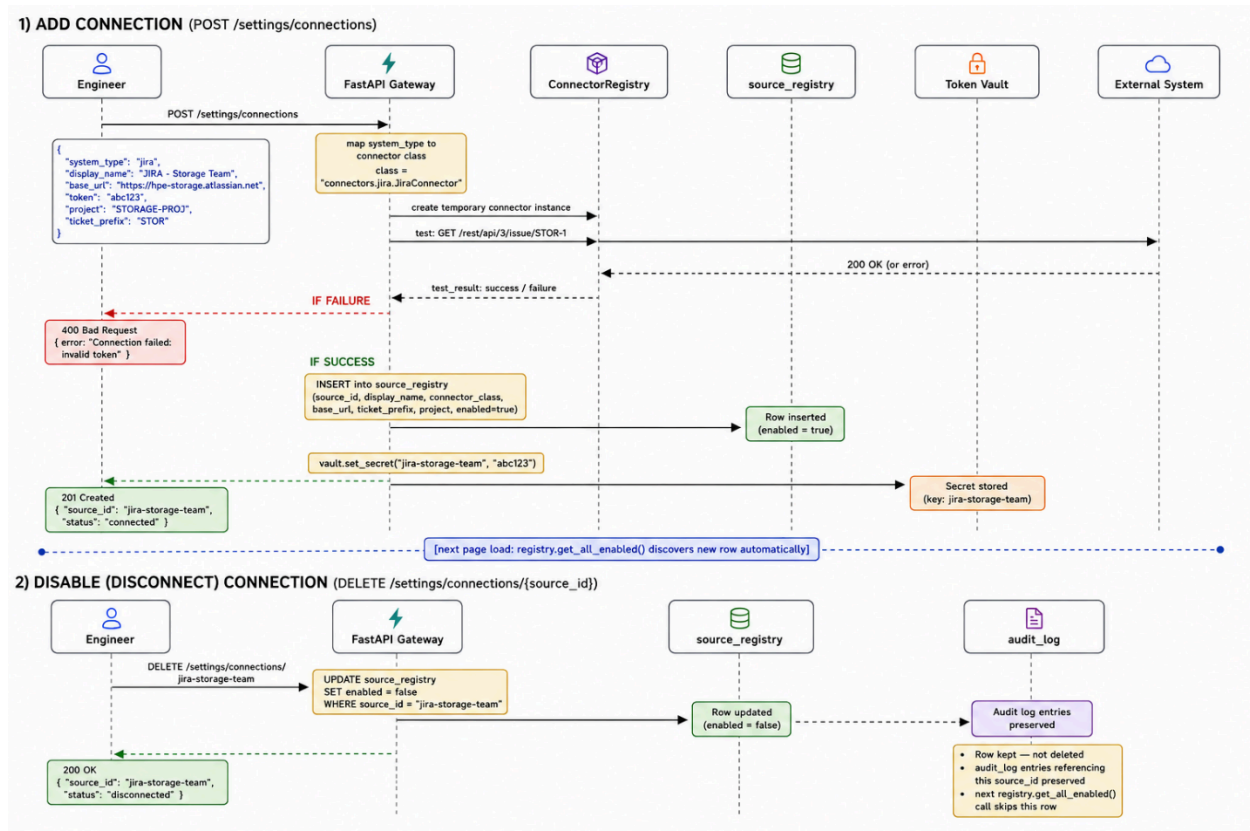
Contains only AI-generated outputs. No bug content from external systems is stored.

2. DELETE the pipeline_context row for this case_id (Working memory is discarded - it was only needed between phases)

3. COMMIT the Kafka consumer offset (Marks the message as fully processed. If this step is skipped, Kafka replays the event on restart)

SD 3: Connector Configuration (Token Settings)

Actors: Engineer, FastAPI Gateway, ConnectorRegistry, source_registry, Token Vault, External System, audit_log



Add Connection (POST /settings/connections):

1. The engineer submits the connection details (system type, display name, base URL, API token, project, and ticket prefix) from the UI.
2. FastAPI maps the selected system type to its corresponding backend connector class (e.g. 'jira' to JiraConnector).
3. FastAPI initializes a temporary, in-memory instance of the connector using the provided configuration.
4. The temporary connector makes a live API validation request to the external system (e.g. GET /rest/api/3/issue/STOR-1) to test credentials and access.
5. If the API call fails, FastAPI aborts the operation, saves nothing, and returns a 400 Bad Request error

containing the validation failure.

6. If validation succeeds, FastAPI inserts the connector configuration into the source_registry database table with the status set to enabled = true.

7. The API token is saved securely in the Token Vault via vault.set_secret() to keep secrets separate from the configuration metadata.

8. On the next page load, the system's registry queries the database for enabled rows and automatically registers the new connector without requiring a restart.

Disable Connection (DELETE /settings/connections/{source_id}):

1. The engineer triggers a disconnect request from the UI, sending a DELETE /settings/connections/{source_id} call to the gateway.

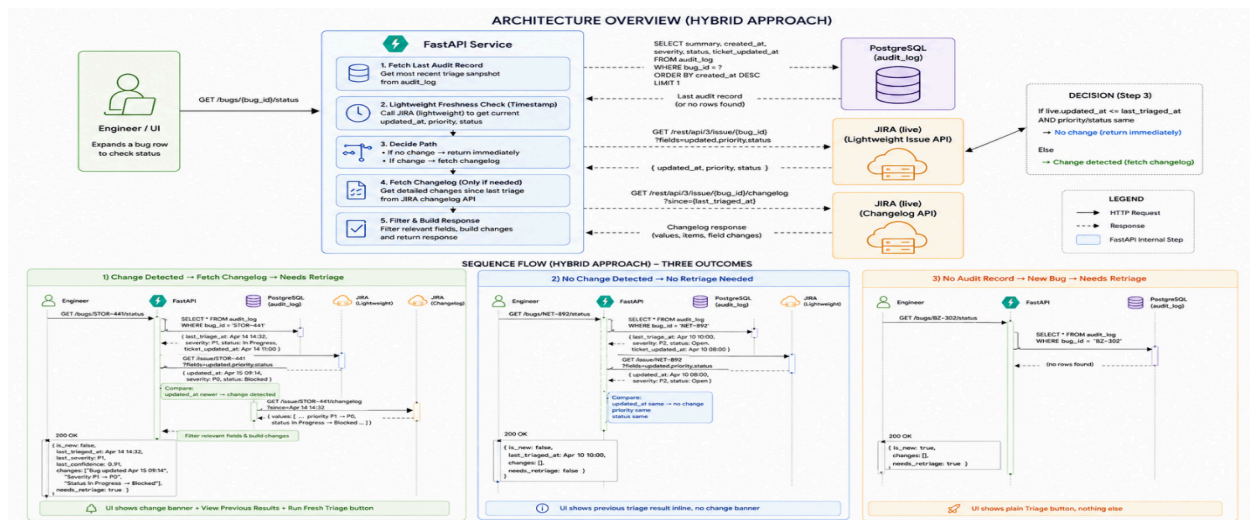
2. FastAPI performs a soft delete by updating the corresponding row in the source_registry database table to enabled = false.

3. The database row is kept intact to maintain foreign key referential integrity and preserve all historical logs referencing this source in the audit_log table.

4. On the next query, the registry filters out the disabled connector using the enabled = true condition, preventing it from executing in future triage runs.

SD 4: Incremental Change Detection - Hybrid Approach

Actors: Engineer/UI, FastAPI Service, PostgreSQL audit_log, JIRA Lightweight Issue API, JIRA Changelog API

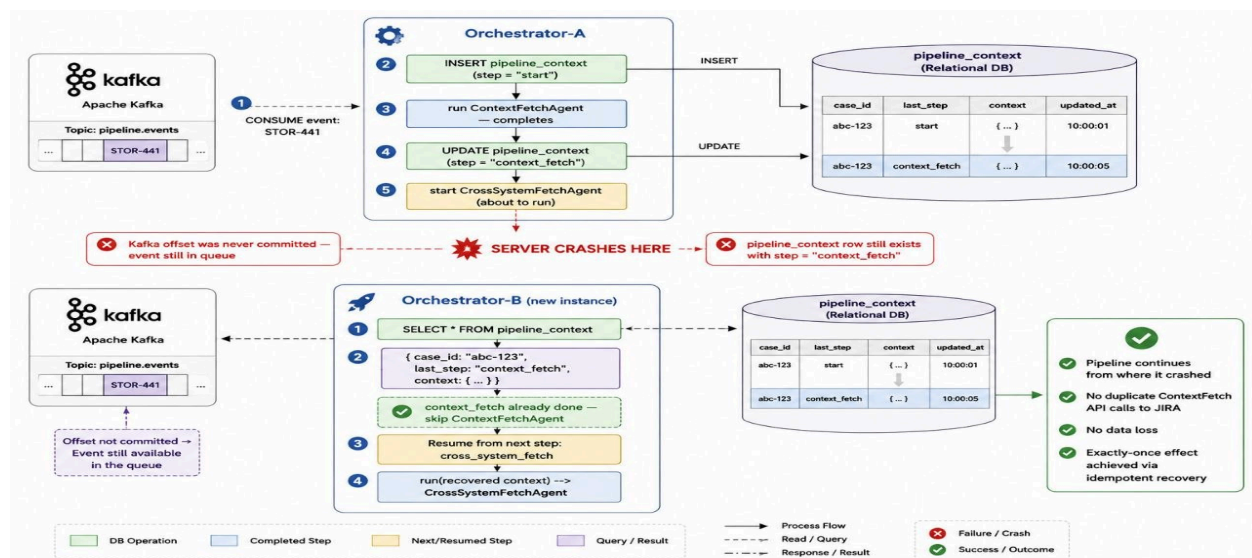


1. The engineer expands a bug row in the UI, which triggers a GET /bugs/{ticket_id}/status request to the FastAPI gateway.

2. FastAPI queries the PostgreSQL audit_log table using: `SELECT summary, created_at, severity, status, ticket_updated_at FROM audit_log WHERE bug_id = ? ORDER BY created_at DESC LIMIT 1`.
3. If no record is found in the database, the system immediately returns `{ is_new: true, needs_retriage: true, changes: [] }`, short-circuiting the flow by bypassing all external API calls, and the UI displays a plain "Triage" button.
4. If an audit record exists, FastAPI calls the external system's lightweight API requesting only three fields: `GET /rest/api/3/issue/{bug_id}?fields=updated,priority,status`.
5. FastAPI compares the returned lightweight fields against the database audit record.
6. If the live update time is less than or equal to the last triaged time, and the priority and status match the database record, the system short-circuits the flow.
7. In this no-change scenario, FastAPI skips the changelog query and immediately returns `{ is_new: false, needs_retriage: false, changes: [] }`, prompting the UI to display the cached previous triage results inline.
8. If the update time is newer, or if the priority or status differs, FastAPI makes an API call to fetch the changelog: `GET /rest/api/3/issue/{bug_id}/changelog?since={last_triaged_at}`.
9. FastAPI processes the returned changelog and filters the field-level history to include only triage-significant fields: priority, status, severity, assignee, and resolution.
10. FastAPI returns the JSON response: `{ is_new: false, needs_retriage: true, changes: [...], last_triaged_at, last_severity, last_confidence }`.
11. The UI receives this response, renders an orange "Changed" banner listing each specific field change, and displays buttons to "View Previous Results" or "Run Fresh Triage".

SD 5: Crash Recovery & Fault Tolerance

Actors: Kafka, Orchestrator-A (crashes), Orchestrator-B (new instance), pipeline_context



Orchestrator-A Phase (Normal Execution Until Crash):

1. Orchestrator-A consumes the triage request event for bug STOR-441 from the Kafka pipeline.events topic without committing the offset.
2. Orchestrator-A inserts a new checkpoint row into the PostgreSQL pipeline_context database table with the status set to step = "start".
3. Orchestrator-A executes the ContextFetchAgent to pull data and keywords from JIRA, which completes successfully.
4. Orchestrator-A updates the database checkpoint row to step = "context_fetch", saving the Phase 1 output in the context JSON column.
5. Orchestrator-A attempts to start the next agent, the CrossSystemFetchAgent.
6. The server crashes mid-execution before the agent finishes.
7. At the time of the crash, the Kafka offset has not been committed, leaving the STOR-441 event in the message queue.
8. The pipeline_context row remains preserved in the PostgreSQL database with the step marked as context_fetch.

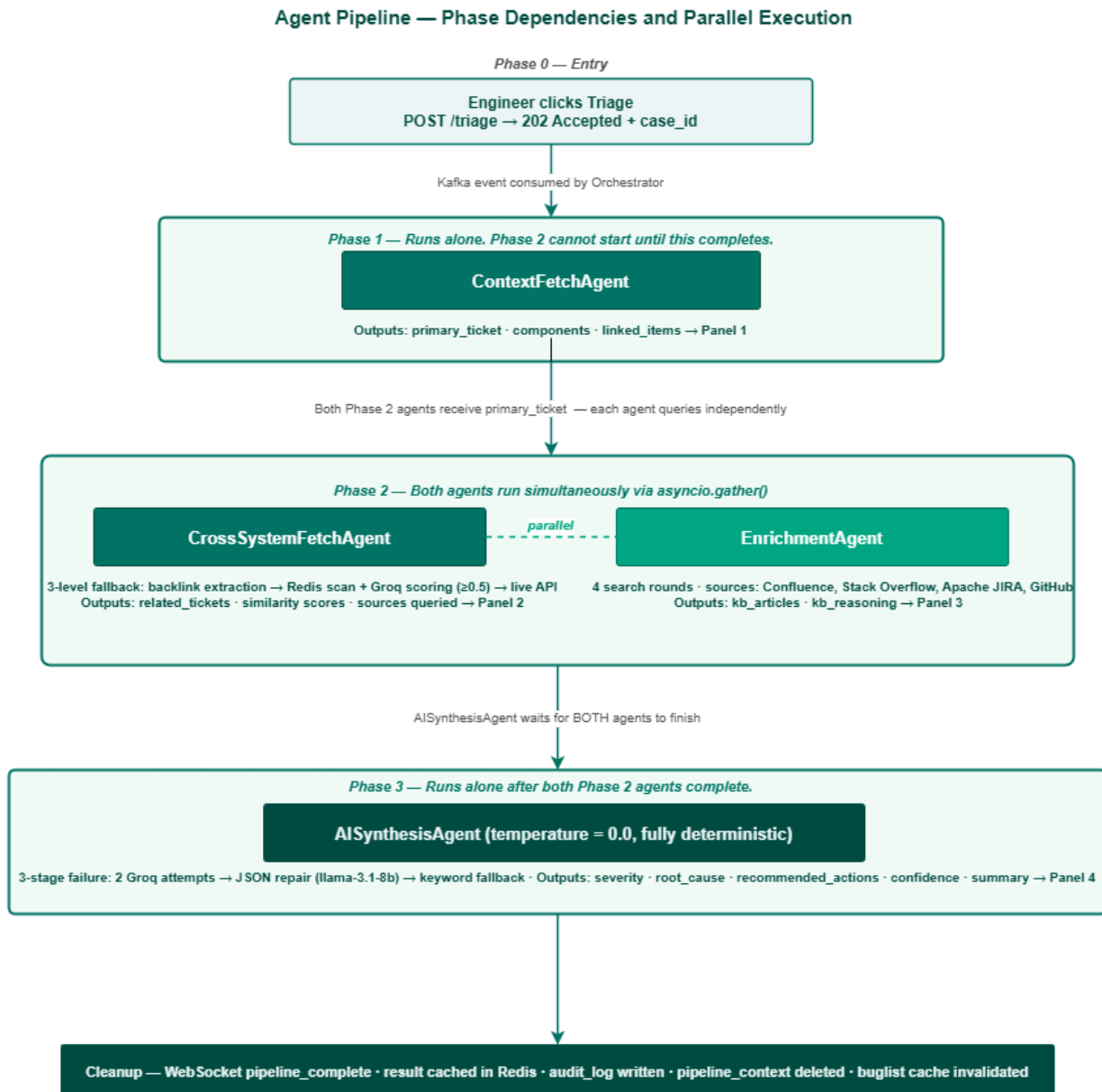
Orchestrator-B Phase (Recovery and Resumption):

1. Orchestrator-B starts up as a new instance and Kafka redelivers the uncommitted STOR-441 triage event to it.
2. Orchestrator-B queries the database using SELECT * FROM pipeline_context to detect any unfinished checkpoints.
3. Orchestrator-B finds the saved checkpoint row matching case_id = "abc-123" with the status last_step = "context_fetch".
4. Orchestrator-B loads the stored context JSON containing all Phase 1 outputs directly from the database.
5. Orchestrator-B identifies that context_fetch is already done, skips executing the ContextFetchAgent entirely, and avoids making duplicate JIRA API calls.
6. Orchestrator-B determines that the next step to run is cross_system_fetch.
7. Orchestrator-B resumes the pipeline from the exact point of failure by calling run(recovered_context) on the CrossSystemFetchAgent.
8. The pipeline completes the remaining execution phases without data loss, duplicate steps, or manual intervention, achieving an exactly-once effect.

8. Agent Pipeline Design

8.1 Pipeline Structure and Phase Ordering

The pipeline has four agents in three sequential phases. The phase ordering is enforced by strict data dependencies. `CrossSystemFetchAgent` and `EnrichmentAgent` both require the complete `TicketData` produced by `ContextFetchAgent` - the full title, description, error excerpt, component, and comments - to generate effective LLM-driven search queries. `AISynthesisAgent` requires the outputs of all three preceding agents to produce meaningful synthesis. Running any agent before its dependencies complete would produce incomplete or poor-quality output.



Phase	Agent	Runs	Delivers
Phase 1	ContextFetchAgent	Alone - everything waits	primary_ticket, components, linked_items - Panel 1
Phase 2a	CrossSystemFetchAgent	Parallel with Phase 2b	related_tickets, sources_queried - Panel 2
Phase 2b	EnrichmentAgent	Parallel with Phase 2a	kb_articles, kb_reasoning - Panel 3
Phase 3	AlSynthesisAgent	Alone - after both Phase 2 agents finish	synthesis output - Panel 4

8.2 Phase 0 - Triggering the Pipeline

The engineer clicks Triage. The browser sends POST /trriage with bug_id and source_id. The gateway validates the JWT, generates a UUID case_id, publishes a Kafka event, and returns 202 Accepted immediately. The browser opens a WebSocket and subscribes to the Redis channel ws:{case_id}. A 1.5-second startup delay at the beginning of the orchestrator's run() method gives the browser time to subscribe before panels start publishing, preventing the race condition. The orchestrator inserts a pipeline_context row as the crash recovery checkpoint.

8.3 Phase 1 - ContextFetchAgent

ContextFetchAgent does two things in sequence. First, it calls connector.get(bug_id) to fetch the complete bug: title, full description with ADF parsing for JIRA, all recent engineer comments, labels, component, assignee, reporter, and timestamps. Second, it calls connector.get_linked_items(bug_id) to fetch any directly cross-referenced bugs. The full primary ticket is passed to both Phase 2 agents who use it directly to generate their own search strategies.

The output is stored in pipeline_context and published as Panel 1. The engineer sees the full bug details appear in about two seconds. A note on JIRA ADF: Atlassian Cloud JIRA returns description as a nested JSON tree. The _extract_text_from_adf() method recursively walks the tree and produces clean plain text before any processing.

Attribute	Detail
Purpose	Retrieve the complete primary ticket with all fields and any directly linked items.
Inputs	bug_id (string), source_id (resolved from ticket prefix if not provided by frontend).
External calls	connector.get(ticket_id). connector.get_linked_items(ticket_id).
Outputs	Full TicketData object, linked_items list. Stored in pipeline_context for use by both Phase 2 agents.
Pipeline context write	Writes {primary_ticket, linked_items} to pipeline_context.context_json JSONB.
WebSocket push	Publishes panel:bug_context to Redis persistent key AND pub/sub channel ws:{case_id}.
Failure handling	If connector.get() fails or times out: sets primary_ticket=None, adds error to context. Pipeline continues with empty context (CrossSystemFetch and Enrichment receive no

	ticket to work with).
--	-----------------------

8.4 Phase 2a - CrossSystemFetchAgent

CrossSystemFetchAgent has three steps.

Step A: the complete primary ticket is sent to Groq which generates a 3-5 keyword search query focused on the core technical concept rather than just title words. For a CTE optimizer bug it generates 'NormalizeCTEIds CTE InlineCTE optimizer' - terms that appear in related bug titles in JIRA.

Step B: that query fires against all other enabled connectors simultaneously via `asyncio.gather()`. Confluence and Customer Portal are excluded.

Step C: all candidates plus the primary ticket are sent to Groq for field-by-field scoring. The scoring prompt asks the LLM to weight error signatures, component match, steps to reproduce, and occurrence context. Candidates below 0.25 are filtered out. If LLM scoring fails, a keyword-overlap fallback scorer runs instead.

Attribute	Detail
Purpose	Find semantically related bugs in all source systems.
Inputs	Full TicketData from Phase 1.
Step A - LLM query generation	Sends all TicketData fields to Groq. Prompt requests a 3-5 keyword semantic search query. temperature=0.
Step B - Parallel search	<code>asyncio.gather()</code> fires <code>connector.search(query, max_results=5)</code> on all other sources simultaneously.
Step C - LLM scoring	Sends primary ticket and all candidates to Groq. Requests <code>similarity_score</code> (0.0-1.0), <code>match_label</code> , <code>match_reason</code> per candidate. temperature=0.
Similarity thresholds	≥ 0.90: Excellent Match ≥ 0.75: Good Match ≥ 0.50: Fair Match < 0.25: filtered out
Outputs	List of scored candidates sorted by <code>similarity_score</code> descending. Panel 2 data.
Failure handling	Individual connector failure returns empty list for that source. LLM scoring failure falls back to keyword-overlap count. Pipeline never errors.

8.5 Phase 2b - EnrichmentAgent

EnrichmentAgent runs a three-round iterative search against Atlassian Confluence Cloud at `cpp3-hpe.atlassian.net/wiki`, space HPEKB. The space contains various technical documentation and troubleshooting guides covering known product issues and architectural compatibility.

Round 1 uses the top three keywords. Round 2 uses the component name. Round 3 uses significant title words after stop-word removal. Results are deduplicated and scored by keyword hit count. A Groq call produces a two-sentence technical reasoning summary. If Confluence returns nothing, the agent falls back to the local `kb_articles` PostgreSQL table.

Attribute	Detail
Purpose	Retrieve relevant KB articles from real Atlassian Confluence Cloud using a multi-round search strategy.
Inputs	Full TicketData from Phase 1 including title, description, component, error excerpt, and comments.
Round 1 search	LLM reads the full ticket and generates an initial CQL search query. Executed via ConfluenceConnector with Basic Auth.
Round 2 search	CQL: space=HPEKB AND text~'{component name}'.
Round 3 search	CQL: space=HPEKB AND text~'{significant title words after stop-word removal}'.
Scoring	Articles scored by count of primary ticket keywords appearing in title + description. High: ≥3 hits. Medium: 1-2. Low: 0.
Reasoning	Groq call: 'which article is most relevant and what specific section should the engineer read'. 2-sentence output.
Fallback	If Confluence API returns 0 results: queries local kb_articles PostgreSQL table using ILIKE search.
Failure handling	Confluence timeout or auth failure: returns empty list. Pipeline continues with reduced context.

8.6 Phase 3 - AISynthesisAgent

AISynthesisAgent assembles the complete context - primary ticket, related tickets with similarity scores and reasons, KB articles, and customer cases - and calls Groq at temperature=0. The response is validated against SynthesisOutput Pydantic model requiring eight fields: unified_severity (must be P0–P3), status_summary, affected_components, root_cause, recommended_actions, engineer_summary, confidence (0.0–1.0), reasoning, and used_fallback. On validation failure, a second attempt adds an explicit schema block to the prompt. On second failure, keyword_fallback() runs producing confidence=0.50 and used_fallback=True with an amber warning in Panel 4.

The SynthesisOutput Pydantic model is defined as follows:

```
class SynthesisOutput(BaseModel):
    unified_severity: str = Field(description="P0, P1, P2, or P3")
    status_summary: str
    affected_components: List[str] = Field(default_factory=list)
    root_cause: str
    recommended_actions: List[str] = Field(default_factory=list)
    summary: str
    confidence: float = Field(ge=0.0, le=1.0)
    reasoning: str
    used_fallback: bool = False #Defaults to False; becomes True when fallback
    synthesis is used.
```

Attribute	Detail
Purpose	Produce a structured triage result from all available context.
Inputs	primary_ticket, related_tickets with scores, kb_articles with relevance,

	customer_cases.
LLM call	Single Groq API call. temperature=0. response_format: json_object. Max tokens: 2000.
Attempt 1	Standard synthesis prompt with full context.
Attempt 2 (on validation failure)	Same prompt plus explicit SynthesisOutput JSON schema appended.
Fallback (both attempts fail)	keyword_fallback() scans title and error_excerpt for severity keywords. Returns confidence=0.50, used_fallback=True.
Outputs	Validated SynthesisOutput object. Panel 4 data.
WebSocket push	Publishes panel:ai_summary to Redis. Publishes pipeline_complete message with severity and confidence.

8.7 Cleanup and Crash Recovery

After Phase 3 the orchestrator inserts one audit_log row containing case_id, bug_id, engineer_id, the full SynthesisOutput as a JSONB summary, systems_queried, and duration_ms. It deletes the pipeline_context row, caches the complete result in Redis for 2 minutes, commits the Kafka offset, and publishes pipeline_complete to the WebSocket channel.

For crash recovery: on restart, the orchestrator finds the existing pipeline_context row and reads current_step. get_steps_to_run(current_step) returns only the steps not yet completed. The context_json column contains all accumulated agent outputs, so the resumed pipeline has everything it needs without re-calling any external API.

9. Implementation

9.1 Backend Implementation

The backend is a Python 3.11 FastAPI application served by Uvicorn with async worker mode. All I/O operations use `async/await` - no thread blocking occurs anywhere in the request path. The application is structured as a set of routers registered on startup, each with its own file and responsibility.

The FastAPI application lifecycle includes two startup events: the Kafka producer is initialised and stored on `app.state`, and the `ConnectorRegistry` loads all enabled connectors from PostgreSQL. Shutdown events flush the Kafka producer and close all database connections cleanly.

```
# api_gateway/main.py
@asynccontextmanager
async def lifespan(app: FastAPI):
    async with kafka_lifespan(app):
        # 1. Kafka producer start
        await ensure_runtime_schema()  # 2. DB schema migration
        consumer_task = asyncio.create_task( # 3. Consumer task
            start_kafka_consumer())
        ingestion_task = asyncio.create_task( # 4. Ingestion loop
            background_ingestion())
    yield
    consumer_task.cancel()
    ingestion_task.cancel()
```

9.2 Kafka Event Processing

The FastAPI gateway produces triage events. The orchestrator consumes them. Both sides use `aiokafka` with manual offset commit.

Topic	Producer	Consumer	Message Key	Retention	Purpose
triage.requests	FastAPI Gateway	TaskOrchestrator	bug_id	7 days	Durable triage request. Uncommitted on crash - replayed on restart. Keyed by <code>bug_id</code> to prevent concurrent duplicate pipeline runs for the same bug.

Producer side in the gateway publishes a JSON payload containing `case_id`, `bug_id`, `source_id`, and `engineer_id`. The orchestrator's Kafka consumer runs in an infinite async loop. It commits the offset only after the complete pipeline finishes, `audit_log` is written, and `pipeline_context` is deleted. This means a crash at any point before that sequence results in the event being redelivered and the pipeline resuming from the last completed step.

```
# orchestrator/kafka_consumer.py
async def consume_loop():
    consumer = AIOKafkaConsumer(
        'triage.requests',
        bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS,
        group_id=KAFKA_CONSUMER_GROUP,
        enable_auto_commit=False,
    )
```

```

async for msg in consumer:
    event = json.loads(msg.value)
    orch = TaskOrchestrator()
    await orch.run(**event)
    await consumer.commit() # only after full pipeline completion

```

9.3 Redis Caching Layer

Redis serves two purposes: a shared cache for bug list data, and a pub/sub bridge for WebSocket panel delivery. The cache uses a key naming convention that encodes the source and filter parameters so different filter combinations do not collide.

Key Pattern	TTL	Contains
buglist:{source_id}:{status}:{severity_or_all}	120 s	Normalised TicketData list from one connector for one status/severity filter.
ticket:{source_id}:{ticket_id}	120 s	Cached ticket detail for one bug to avoid duplicate full-fetch calls.
panel:{case_id}:{panel_name}	120 s	Serialised panel data for short WebSocket replay.
panels:{case_id}	120 s	Ordered list of panel names published for the case.
case:{case_id}	120 s	Temporary complete triage result cache. PostgreSQL remains the source of truth.
triage_result:{source_id}:{bug_id}	120 s	Short-lived per-bug synthesis cache.
related:{case_id}	120 s	Related tickets snapshot for temporary recovery.
enrichment:{case_id}	120 s	Enrichment sources snapshot for temporary recovery.
kb:{case_id}	120 s	KB articles snapshot for temporary recovery.
lock:ingest:all	60 s	Distributed ingestion lock to prevent concurrent refresh.
ws:{case_id}	No TTL	Redis Pub/Sub channel for live WebSocket delivery.

If Redis is unavailable, all `cache.get()` calls return `None` and the connector proceeds directly to the external API. Redis unavailability degrades performance but does not prevent the pipeline from functioning.

9.4 AI Agent Implementation Pattern

All agents follow the same implementation pattern. They extend `BaseAgent`, which provides the `_add_error()` helper and the `step_name` class attribute. The orchestrator calls `agent.run(context)` and receives the updated context. Agents never raise exceptions - all errors are caught and recorded in `context['errors']` so the pipeline always continues.

```

# orchestrator/agents/base.py
class BaseAgent(ABC):
    step_name: str = 'base'

    @abstractmethod
    async def run(self, context: dict) -> dict:

```

```

raise NotImplementedError

async def safe_run(self, context: dict) -> tuple[dict, int]:
    start = time.monotonic()
    try:
        context = await self.run(context)
    except Exception as e:
        self._add_error(context, f'{self.step_name} failed: {str(e)}')
    duration_ms = int((time.monotonic() - start) * 1000)
    return context, duration_ms

def _add_error(self, context: dict, error: str) -> None:
    errors = context.get('errors', {})
    errors[self.step_name] = error
    context['errors'] = errors

```

9.5 Reliability and Graceful Degradation

The system is designed so that a result is always delivered to the engineer regardless of individual component failures. Every failure scenario has a defined degraded behaviour.

Failure Scenario	Degraded Behaviour	Engineer Experience
One connector unavailable	Remaining connectors serve results. Failed source records error in <code>systems_queried</code> .	Partial related issues in Panel 2. Other panels unaffected.
Confluence API unavailable	EnrichmentAgent returns empty list. Falls back to local <code>kb_articles</code> DB.	Panel 3 shows local articles or is empty. Lower confidence in Panel 4.
LLM API rate limited	AISynthesisAgent retries once, then runs <code>keyword_fallback()</code> .	Panel 4 shows result with amber 'low confidence' banner and <code>used_fallback=True</code> .
Redis unavailable	Cache reads return None. All fetches go directly to external APIs.	Higher latency. Rate limit risk. No functional impact.
Kafka unavailable	<code>ENABLE_LOCAL_PIPELINE_FALLBACK</code> runs orchestrator as asyncio task in-process.	Triage works. Crash recovery disabled until Kafka recovers.
Orchestrator crashes mid-pipeline	Kafka replays event. Orchestrator reads <code>pipeline_context</code> and resumes from last completed step.	Brief delay. Transparent to engineer. Result delivered on resume.
PostgreSQL <code>audit_log</code> write fails	Pipeline result already delivered via WebSocket. Audit write retried.	Engineer has result. Audit record may appear with a short delay.

10. API Design

10.1 Authentication Endpoints

Meth od	Endpoint	Query / Body	Response
POST	/auth/login	{ email, password }	{ access_token, token_type: 'bearer', role, user_id, expires_in }
GET	/auth/me	-	{ user_id, email, role, display_name }

10.2 Bug Discovery and Triage Endpoints

Meth od	Endpoint	Query / Body	Response
GET	/bugs	page, page_size, severity, source, status, search, sort_field, sort_order	{ bugs[], total, sources_online, partial }
POST	/bugs/refresh	-	{ status, connectors_cleared }
POST	/bugs/warm	-	{ status, message }
GET	/bugs/{bug_id}/status	-	{ is_new, needs_retriage, changes[], last_triaged_at, last_severity, last_confidence }
POST	/trriage	{ bug_id, source_id }	{ case_id, bug_id, source_id, status: 'accepted' }
WS	/trriage/{id}/stream	- (token query param)	{ panel, data } messages then { type: 'pipeline_complete' }
GET	/trriage/{case_id}/result	-	Full triage result from Redis or audit log

10.3 History and Metrics Endpoints

Meth od	Endpoint	Query / Body	Response
GET	/metrics	-	{ total_triaged, by_severity, source_counts, avg_confidence, live_p0, live_p1, needs_triage, recent_activity[] }
GET	/history/trriage	limit (default 50, max 200)	{ entries[] with case_id, bug_id, source_id, status, summary, created_at }

10.4 Settings - Connector Management Endpoints

Meth od	Endpoint	Query / Body	Response
------------	----------	--------------	----------

GET	/settings/connections	-	{ connections[] with source_id, display_name, system_type, enabled, health, token_present }
POST	/settings/connections	{ display_name, system_type, base_url, project_key, ticket_prefix, token }	{ source_id, display_name, system_type, health }
PUT	/settings/connections/{source_id}	{ display_name?, base_url?, project_key?, ticket_prefix?, enabled?, token? }	{ source_id, updated_fields }
DELETE	/settings/connections/{source_id}	-	{ status, source_id, message }
PUT	/connections/{source_id}	{ display_name?, base_url?, project_key?, ticket_prefix?, token? }	{ status, source_id } - Legacy endpoint
POST	/settings/connections/{source_id}/test	-	{ ok, status, latency_ms, error? }

10.5 Cases Endpoints

Meth od	Endpoint	Query / Body	Response
GET	/cases/{case_id}	-	{ case_id, primary_ticket, related_tickets, kb_articles, synthesis, enrichment }

11. Frontend Design

11.1 Technology Overview

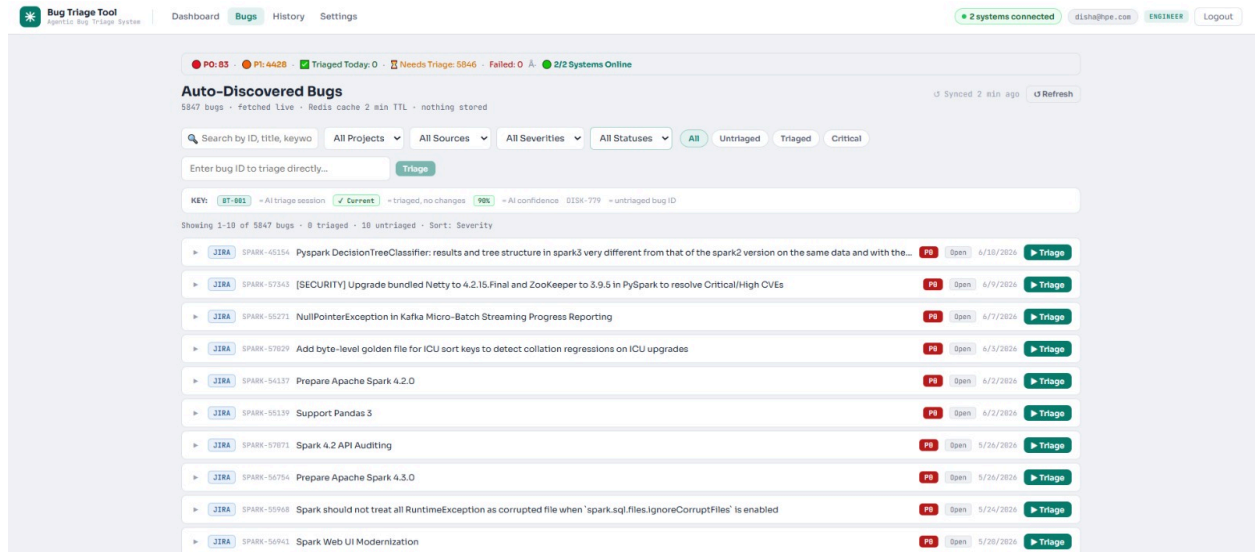
- Built as a Single-Page Application (SPA) using React 18 for component-based rendering and Vite 5 for hot module replacement (HMR) and optimized production builds.
- Routing is handled dynamically via React Router v6, ensuring smooth page transitions and sidebar navigation without full browser reloads.
- Server state caching, API data synchronization, and automated query retries are managed using TanStack Query (React Query) to optimize network performance.
- HTTP communications use the native Web Fetch API wrapped inside a centralized [apiFetch](#) utility ([api/client.js](#)) which automatically injects JWT authorization headers and handles session-expiration redirects.
- User session tokens (JWT) are stored securely in [sessionStorage](#) rather than [localStorage](#) to ensure credentials are automatically cleared when the user closes their browser tab.

11.2 Pages

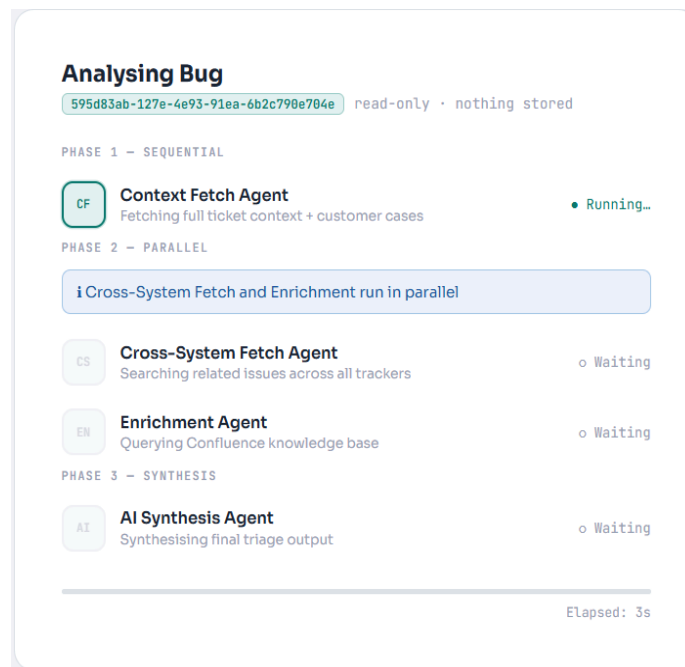
Page	Route	What the Engineer Sees
Login	/login	Email and password form. Demo credentials shown below the form. On success, JWT stored in localStorage and user redirected to /bugs.
Bug List	/bugs	Dashboard strip at top showing live P0 and P1 counts, triaged today, and systems online. Filter bar for severity, source, and status. Search box. Bug list with skeleton loading rows that appear immediately while data loads. Expandable rows for change detection. Triaged bugs shown as expandable groups with related issues as children.
Triage	/triage/:caseId	STATE 1 while running: four agent cards showing active phase, a progress bar filling as panels arrive, and an elapsed timer. STATE 2 after completion: 2x2 grid of four panels - Bug Context, Related Issues, Knowledge Base, AI Summary. If loaded from History via ?from=history, cached result is loaded directly without opening a WebSocket.
History	/history	Table of all past triages from audit_log. Columns: bug ID, source, AI severity, confidence %, root cause excerpt, duration, triaged-at timestamp. View Results and Re-triage buttons on each row.
Dashboard	/dashboard	Five stat cards: P0 Critical, P1 High, Triaged Today, Needs Triage, Systems Online. Recent activity table from audit_log last ten entries.
Settings	/settings	Left sidebar filtering by system type. Right panel showing connector cards with status indicator, Test button, and Remove button. Add Connection button opens a modal form. Admin-only User Management section below connectors.

11.3 WebSocket Connection Management

The WebSocket opens immediately after POST /triage returns the case_id. The client processes three message types: a panel message updates the corresponding panel and advances the progress bar; a pipeline_complete message switches the page from loading to results state; an error message shows an error panel and stops the animation. The client reconnects automatically up to two times with a 1.5-second gap if the connection drops. A 90-second hard timeout shows a timeout message if no pipeline_complete arrives.



[Figure 9 - Bug List page with dashboard strip and filter bar]



[Figure 10 - Triage page loading state with agent progress cards]

-- Back to Bugs
BT-F75CA
SPARK-57029
Add byte-level golden file for ICU sort keys to detect collation regressions on ICU upgrades
21s
JIRA

01 Bug Context

Ticket ID: SPARK-57029
Source: Apache Spark (JIRA)
Title: Add byte-level golden file for ICU sort keys to detect collation regressions on ICU upgrades
Status: Open
Severity: P0
Component: SQL
Assignee: Kent Yao
Reporter: Kent Yao
Updated: 6/3/2026, 9:16:00 AM
Created: 5/23/2026
URL: Open source ticket

DESCRIPTION

h3. Motivation ICU upgrade PRs (SPARK-50189 76.1 / SPARK-52038 77.1 / SPARK-54447 78.1 / SPARK-55308 78.2 / SPARK-56397 78.3) currently only touch the dependency file and ([ICUCollationsMapSuite]). Sort-key byte changes between ICU versions go undetected without a review-trigger snapshot. Spark relies on ICU4J's ([Collator.getCollationKey(...).toByteArray()]) to compare binary-comparable sort keys for ICU collations. These byte sequences are silently versioned

02 Related Issues

5 found

JIRA SPARK-50189 SPARK-50189
68% Referenced
Referenced in ticket text (unverified)

JIRA SPARK-52038 SPARK-52038
68% Referenced
Referenced in ticket text (unverified)

JIRA SPARK-54447 SPARK-54447
68% Referenced
Referenced in ticket text (unverified)

JIRA SPARK-55308 SPARK-55308
68% Referenced
Referenced in ticket text (unverified)

03 Knowledge Base

1 results

8 medium

Add byte-level golden file for ICU sort keys to detect collation regressions on ICU upgrades

h3. Motivation ICU upgrade PRs (SPARK-50189 76.1 / SPARK-52038 77.1 / SPARK-54447 78.1 / SPARK-55308 78.2 / SPARK-56397 78.3) currently only touch the dependen...

04 AI Summary

98% confidence

P0

90% AI confidence score

ROOT CAUSE

ICU library upgrades can change sort-key byte sequences without detection, potentially causing collation regressions

RECOMMENDED ACTIONS

01. Add byte-level golden files for ICU sort keys to detect collation regressions
02. Review and test ICU upgrade PRs to ensure sort-key byte sequences remain consistent
03. Implement automated testing to catch changes in sort-key byte sequences

SUMMARY

The bug is related to ICU library upgrades changing sort-key byte sequences without detection, potentially causing collation

[Figure 11 - Triage page results state showing all four panels]

FILTER BY SYSTEM

All Systems14

GitHub5

Apache JIRA5

Bugzilla1

Confluence2

Customer Portal1

All Systems

+ Add Connection

Connectors seeded via init_db.py on startup. Changes take effect immediately.

GH

Apache Spark (GitHub)SGH

https://api.github.com/apache/spark

ConnectedTestRemove

J

Apache Spark (JIRA)SPARK

https://issues.apache.org/jira/SPARK

Internal/PublicTestRemove

GH

Apache Kafka (GitHub)KGH

https://api.github.com/apache/kafka

ConnectedTestRemove

J

Apache Kafka (JIRA)KAFKA

https://issues.apache.org/jira/KAFKA

Internal/PublicTestRemove

BZ

Mozilla Firefox (Bugzilla)BUG

https://bugzilla.mozilla.org/Firefox

Internal/PublicTestRemove

Apache Flink — GitHub IssuesFGH

[Figure 12 - Settings page showing connector cards and Add Connection model]

62

12. Security Design

12.1 JWT Authentication

All API requests except `/auth/login` and `/health` require a valid JWT in the Authorization header. The JWT payload contains three claims: `sub` (user email), `role`, and `exp` (expiry, default 120 minutes). The signing secret is stored only as an environment variable. The server is stateless - there is no session database. Token validation is signature checking plus expiry checking on every request.

12.2 Role-Based Access Control

Role	Bug List	Triage Result	History	Metrics	Settings
Engineer	Full access	Full 4 panels with all technical detail	Own triages only	Full access	Full CRUD

12.3 OpenBAO Token Vault

API credentials are stored in OpenBAO, the community-maintained open-source fork of HashiCorp Vault recommended by HPE mentors (MPL 2.0 licence). OpenBAO encrypts secrets at rest with AES-256-GCM, logs every read with requestor identity and timestamp, and supports token rotation without restart. The `TokenVault` class provides `get_secret(path, fallback_env)`: it queries OpenBAO and falls back to environment variables if the vault is unavailable. The development environment works without OpenBAO while production uses it with no code changes.

12.4 Data Privacy by Architecture

The most important security property is an architectural guarantee rather than a policy control. Bug content from external systems cannot leak from a database breach because it is never written to any permanent storage. `pipeline_context` holds it for a few seconds and then deletes it. `audit_log` never receives it. Redis cache expires in five minutes. An attacker who gains full read access to the production database sees only connector configuration, AI severity labels, and triage timestamps.

13. Proof of Concept Demonstration Plan

13.1 Demonstration Scenarios

Scenario	What Will Be Shown	Demo Data
Scenario 1 - Live Bug List Auto-Discovery:	Open the platform with no manual input. The bug list auto-populates from all configured connectors in parallel. Show bugs from Apache Spark GitHub and Apache Spark JIRA appearing in the same unified list sorted by severity.	apache-spark-github + apache-spark-jira
Scenario 2 - Cross-System Correlation:	Triage an Apache Spark bug. Show the Related Issues panel finding the same bug tracked in both JIRA and GitHub with similarity scores and match reasons.	SPARK-56468 primary, apache-kafka-github secondary
Scenario 3 - Confluence KB Enrichment	Show the Linked Context panel returning real pages from the Confluence knowledge base with direct links to the relevant section.	HPE Confluence HPEKB space
Scenario 4 Full AI Synthesis	Show Panel 4 with structured severity, root cause, recommended actions, and confidence score. Run the same bug twice to demonstrate temperature=0 determinism - same output both times.	Groq LLM, temperature=0
Scenario 5 - Incremental Change Detection	Triage a bug, then expand the row after its status has changed externally. Show the specific change delta with the Re-triage button.	JIRA changelog API
Scenario 6 - Dynamic Connector Configuration	Add a new GitHub repository via the Token Settings UI. Show the Test Connection step passing, then show the new connector's bugs appearing in the list on the next page load with no server restart.	microsoft/vscode-github added live

14. Data Sources

Because HPE internal bug tracking systems require confidential credentials unavailable to this project, publicly accessible open-source projects serve as demonstration data. These were selected specifically to include projects tracked across more than one bug system, making cross-system correlation demonstrable with real data.

source_id	System	Project	Unresolved Bugs	Why Selected
apache-spark-github	GitHub Issues	apache/spark	~5,800	Same project as Spark JIRA. Enables the primary cross-system demo.
apache-spark-jira	Apache JIRA v2	SPARK	~1,200	Public, no credentials required. Used by CrossSystemFetchAgent for cross-system correlation.
apache-kafka-github	GitHub Issues	apache/kafka	~1,400	Large active project with diverse bug types.
apache-kafka-jira	Apache JIRA v2	KAFKA	~900	Same project as Kafka GitHub.
apache-flink-github	GitHub Issues	apache/flink	~1,800	Third large Apache data processing project.
apache-flink-jira	Apache JIRA v2	FLINK	~900	Same project as Flink GitHub.
apache-hadoop-jira	Apache JIRA v2	HADOOP	~1,500	Classic large-scale open-source project with long history.
apache-hive-jira	Apache JIRA v2	HIVE	~800	SQL-on-Hadoop - closely related to Spark SQL issue types.
kubernetes-github	GitHub Issues	kubernetes/kubernetes	~3,000	Highest-volume active GitHub project.
vscode-github	GitHub Issues	microsoft/vscode	~5,000	Diverse component range useful for testing.
mozilla-firefox-bugzilla	Bugzilla REST	Firefox	~8,000	The only Bugzilla connector. Essential for three-tracker-type demo.

The Atlassian Confluence knowledge base used by EnrichmentAgent is hosted at [cpp3-hpe.atlassian.net/wiki](#) in the HPE Engineering KB space (HPEKB). This space contains a variety of technical documentation and troubleshooting guides-these are real documentation pages, not

placeholder content-and the Confluence search returns genuinely relevant results when the matching bug type is triaged.

15. Conclusion and Future Work

15.1 What This System Delivers

This project delivers a proof of concept demonstrating all core capabilities specified by HPE mentors. The platform connects to all configured bug tracking sources across multiple tracker types, shows a live unified bug list to engineers, and executes a four-phase AI pipeline delivering structured results in under ten seconds. The cross-system correlation engine automatically finds the same bug tracked in different systems and surfaces the connection to the engineer, including status differences between them.

The Confluence integration calls the Atlassian REST API directly, searches with CQL, and returns real pages from a real Confluence Cloud space. Data sovereignty is enforced architecturally rather than by policy. Crash recovery is built into the pipeline: a mid-pipeline failure followed by a restart causes the pipeline to resume from the correct step with the same final output.

15.2 Current Limitations

- Both CrossSystemFetchAgent and EnrichmentAgent use LLM-generated semantic search queries rather than static keyword matching. However, the retrieval layer underneath still uses text-based search (GitHub search API, JIRA JQL text search, Confluence CQL). Upgrading to vector embeddings with pgvector would improve recall for vocabulary mismatches but requires a data sovereignty decision about embedding third-party content.
- The Customer Portal connector uses mock data from a local table. A production version requires real HPE Customer Portal credentials.
- The system uses public open-source data. Connecting to HPE internal JIRA and Bugzilla instances requires additional connector configuration and authentication setup.

References

- [1] S. Tiangolo, "FastAPI - Modern, Fast Web Framework for Building APIs with Python," FastAPI Documentation, 2024. Available: <https://fastapi.tiangolo.com>
- [2] S. Tiangolo, "Pydantic v2 - Data Validation Using Python Type Hints," Pydantic Documentation, 2024. Available: <https://docs.pydantic.dev>
- [3] M. Bayer, "SQLAlchemy 2.0 - The Python SQL Toolkit and Object Relational Mapper," SQLAlchemy Documentation, 2024. Available: <https://docs.sqlalchemy.org/en/20/>
- [4] M. Bayer, "Alembic - Database Migrations for SQLAlchemy," Alembic Documentation, 2024. Available: <https://alembic.sqlalchemy.org/>
- [5] aio-lib, "aiokafka - AsyncIO Client for Apache Kafka," aiokafka Documentation, 2024. Available: <https://aiokafka.readthedocs.io/>
- [6] Apache Software Foundation, "Apache Kafka Documentation," Apache Kafka, 2024. Available: <https://kafka.apache.org/documentation/>
- [7] Redis Ltd., "Redis Documentation," Redis 7, 2024. Available: <https://redis.io/docs/>
- [8] andymccurdy, "redis-py - The Python Interface to the Redis Key-Value Store," PyPI / GitHub, 2024. Available: <https://github.com/redis/redis-py>
- [9] The PostgreSQL Global Development Group, "PostgreSQL 15 Documentation," 2024. Available: <https://www.postgresql.org/docs/15/index.html>
- [10] Groq, "Groq API Documentation," Groq Cloud, 2024. Available: <https://console.groq.com/docs>
- [11] Meta AI, "Llama 3.3 70B Instruct - Model Card," Hugging Face, Dec. 2024. Available: <https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct>
- [12] Atlassian, "Jira Cloud Platform REST API v3," Atlassian Developer Documentation, 2024. Available: <https://developer.atlassian.com/cloud/jira/platform/rest/v3/intro/>
- [13] Atlassian Community, "How can I get the changelog/history for issues via API calls?", Atlassian Community Q&A, 2024. Available: <https://community.atlassian.com/forums/Jira-questions/How-can-I-get-the-changelog-history-for-issues-via-API-calls/qaq-p/2643173>
- [14] Atlassian, "Confluence Cloud REST API," Atlassian Developer Documentation, 2024. Available: <https://developer.atlassian.com/cloud/confluence/rest/>
- [15] GitHub, Inc., "GitHub REST API Documentation," GitHub Docs, 2024. Available: <https://docs.github.com/en/rest>
- [16] GitHub, Inc., "Issue Events REST API," GitHub REST API Documentation, 2026. Available: <https://docs.github.com/en/rest/issues/events?apiVersion=2026-03-10>
- [17] Mozilla Foundation, "Bugzilla REST API," Bugzilla Documentation, 2024. Available: <https://bugzilla.readthedocs.io/en/latest/api/>
- [18] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015. Available: <https://datatracker.ietf.org/doc/html/rfc7519>
- [19] Docker, Inc., "Docker Documentation," 2024. Available: <https://docs.docker.com>

[20] Docker, Inc., "Docker Compose Documentation," 2024. Available:
<https://docs.docker.com/compose/>

[21] Anthropic, "Model Context Protocol (MCP) - Introduction," Model Context Protocol Documentation, 2024. Available: <https://modelcontextprotocol.io/docs/getting-started/intro>

Appendix A - Glossary

Term	Meaning
ADF	Atlassian Document Format. The nested JSON structure Atlassian Cloud JIRA uses for description fields.
Agent	A stateless Python class that receives a context dictionary, does its work, and returns an updated context dictionary.
audit_log	The PostgreSQL table recording the AI's outputs after each triage. Never contains bug content from external systems.
BaseConnector	The abstract Python class all connector implementations must extend.
Cache-Aside	Check cache first, call source only on miss, write to cache, serve future reads from cache.
Case ID	UUID generated per triage request. Ties together Kafka event, pipeline_context row, WebSocket channel, and Redis result.
ConnectorRegistry	Module-level singleton loading connector instances from source_registry and providing them to agents.
CQL	Confluence Query Language. The query syntax for Confluence REST API content search.
Data Sovereignty	Bug content from external systems is never stored permanently in this system's own database.
JWT	JSON Web Token. Signed bearer token containing user email, role, and expiry.
Kafka Offset	Pointer to which Kafka message has been fully processed. Not committed until the full pipeline completes.
pipeline_context	PostgreSQL row storing accumulated pipeline state between agent phases. Deleted on completion.
ReAct	Reason and Act. LLM decides what tool to call, calls it, reads the result, decides whether to continue.
RBAC	Role-Based Access Control. JWT - JSON Web Token containing engineer identity and expiry, used for authentication.
Soft Delete	Setting enabled=false on a row rather than deleting it, preserving the row for audit integrity.
source_registry	PostgreSQL table with one row per connected system.
SynthesisOutput	Pydantic model validated output of AISynthesisAgent.
temperature=0	LLM setting eliminating randomness. Same input always produces same output.
TicketData	Common Python dataclass all connectors normalise to. Same schema regardless of source.
WebSocket	Persistent two-way browser-server connection for streaming panel results progressively.

Appendix B - Full API Reference

Meth od	Endpoint	Query / Body	Response
POST	/auth/login	{ email, password }	{ access_token, token_type: 'bearer', role, user_id, expires_in }
GET	/auth/me	-	{ user_id, email, role, display_name }
GET	/bugs	page, page_size, severity, source, status, search, sort_field, sort_order	{ bugs[], total, sources_online, partial }
POST	/bugs/refresh	-	{ status, connectors_cleared }
POST	/bugs/warm	-	{ status, message }
GET	/bugs/{bug_id}/status	-	{ is_new, needs_retriage, changes[], last_triaged_at, last_severity, last_confidence }
POST	/trriage	{ bug_id, source_id }	{ case_id, bug_id, source_id, status: 'accepted' }
WS	/trriage/{id}/stream	- (token query param)	{ panel, data } messages then { type: 'pipeline_complete' }
GET	/trriage/{case_id}/result	-	Full triage result from Redis or audit log
GET	/metrics	-	{ total_triaged, by_severity, source_counts, avg_confidence, live_p0, live_p1, needs_triage, recent_activity[] }
GET	/history/trriage	limit (default 50, max 200)	{ entries[] with case_id, bug_id, source_id, status, summary, created_at }
GET	/settings/connections	-	{ connections[] with source_id, display_name, system_type, enabled, health, token_present }
POST	/settings/connections	{ display_name, system_type, base_url, project_key, ticket_prefix, token }	{ source_id, display_name, system_type, health }
PUT	/settings/connections/{source_id}	{ display_name?, base_url?, project_key?, ticket_prefix?, enabled?, token? }	{ source_id, updated_fields }
DELETE	/settings/connections/{source_id}	-	{ status, source_id, message }
PUT	/connections/{source_id}	{ display_name?, base_url?, project_key?, ticket_prefix?, token? }	{ status, source_id } - Legacy endpoint
POST	/settings/connections/{source_id}/test	-	{ ok, status, latency_ms, error? }

Appendix C - Environment Variables

Variable	Purpose	Example Value
JWT_SECRET	Signing key for HS256 JWT tokens	any-long-random-string
JWT_ALGORITHM	JWT signing algorithm	HS256
JWT_EXPIRE_MINUTES	Token validity in minutes	120
POSTGRES_URL	PostgreSQL async connection string	postgresql+asyncpg://postgres:postgres@localhost:5432/hpe_bugtriage
REDIS_URL	Redis connection string	redis://localhost:6379/0
REDIS_TTL_TICKET_SECONDS	TTL for individual ticket cache entries (used by Redis client)	300
REDIS_TTL_BUGLIST_SECONDS	TTL for bug list cache entries (used by Redis client)	120
KAFKA_BOOTSTRAP_SERVERS	Kafka broker address	localhost:9092
KAFKA_TOPIC_TRIAGE_REQUESTS	Kafka topic name for triage pipeline events	triage.requests
KAFKA_CONSUMER_GROUP	Kafka consumer group ID for the orchestrator	bugtriage-orchestrator
GROQ_API_KEY	API key for Groq LLM inference (Llama 3.3 70B)	(your Groq key - no fallback; must be set)
GROQ_MODEL	Groq model identifier used by AI Synthesis and Enrichment agents	llama-3.3-70b-versatile
CONFLUENCE_EMAIL	Email for Confluence REST API authentication	user@domain.com
CONFLUENCE_URL	Base URL for the Confluence instance	https://cpp3-hpe.atlassian.net/wiki
ENABLE_LOCAL_PIPELINE_FALLBACK	Run pipeline locally if Kafka is unavailable (dev only)	true
OpenBAO / auth_secret_ref keys	Runtime-managed secrets for connector auth tokens (stored in OpenBAO, not .env)	managed at runtime via /settings API

Appendix D - Database Schemas

source_registry

```
CREATE TABLE source_registry (  
    source_id      TEXT PRIMARY KEY,  
    display_name   TEXT NOT NULL,  
    system_type    TEXT NOT NULL,  
    base_url       TEXT NOT NULL,  
    port           INTEGER,  
    auth_type      TEXT NOT NULL DEFAULT 'bearer_token',  
    auth_secret_ref TEXT,  
    project_key    TEXT,  
    ticket_prefix  TEXT,  
    enabled        BOOLEAN DEFAULT TRUE,  
    created_at     TIMESTAMPTZ DEFAULT now()  
);
```

pipeline_context

```
CREATE TABLE pipeline_context (  
    case_id      TEXT PRIMARY KEY,  
    current_step TEXT DEFAULT 'start',  
    context_json JSONB,  
    created_at   TIMESTAMPTZ DEFAULT now(),  
    updated_at   TIMESTAMPTZ DEFAULT now()  
);  
-- Rows deleted by the orchestrator immediately on pipeline completion.
```

audit_log

```
CREATE TABLE audit_log (  
    id          SERIAL PRIMARY KEY,  
    case_id     TEXT,  
    bug_id      TEXT NOT NULL,  
    source_id   TEXT,  
    engineer_id TEXT,  
    step        TEXT NOT NULL,  
    status      TEXT DEFAULT 'done',  
    summary     JSONB,  
    systems_queried JSONB,  
    duration_ms INTEGER,  
    created_at  TIMESTAMPTZ DEFAULT now()  
);  
-- CONSTRAINT: this table must never contain bug titles, descriptions,  
-- comments, error logs, or any content from external systems.
```

user_roles

```
CREATE TABLE user_roles (  
    user_id      TEXT PRIMARY KEY,  
    role         TEXT NOT NULL,  
    password_hash TEXT NOT NULL,  
    display_name TEXT,  
    created_at   TIMESTAMPTZ DEFAULT now()  
);
```

system_group_registry

```
CREATE TABLE system_group_registry (  

```

```
group_id          TEXT PRIMARY KEY,
status            TEXT DEFAULT 'active',
priority          TEXT,
title             TEXT,
primary_source_id TEXT,
created_at        TIMESTAMPTZ DEFAULT now(),
updated_at        TIMESTAMPTZ DEFAULT now()
expires_at        TIMESTAMPTZ DEFAULT (now() + INTERVAL '30 minutes')
);
```

bug_group_mappings

```
CREATE TABLE bug_group_mappings (
  id              SERIAL PRIMARY KEY,
  group_id        TEXT NOT NULL,
  raw_ticket_id   TEXT NOT NULL,
  source_id       TEXT NOT NULL,
  system_type     TEXT,
  role            TEXT NOT NULL DEFAULT 'child',
  similarity_score FLOAT,
  similarity_label TEXT,
  similarity_reason TEXT,
  created_at      TIMESTAMPTZ DEFAULT now(),

  CONSTRAINT uq_bug_group_ticket UNIQUE (raw_ticket_id, source_id)
  CONSTRAINT fk_group FOREIGN KEY (group_id)
    REFERENCES system_group_registry(group_id)
    ON DELETE CASCADE
);
```